4th Year

Architecture study for cross-document consistency analysis via dependency graphs

Qualipro®

Mr. Mohamed Sahbi BEN KALAIA

Technical Director, Qualipro

Company Supervisor	Internship Committee Review
--------------------	-----------------------------

Academic Year: 2025 / 2026

Acknowledgements

I would like to express my sincere gratitude to Mr. Kamel GAZZEH, Technical Director at Qualipro, for supervising this internship. His guidance, his availability and the trust he placed in me gave me the freedom to explore the subject in depth, while his remarks kept the work focused on what matters in practice.

My thanks also go to the whole Qualipro team, for their warm welcome and for the friendly and stimulating atmosphere I found from my first days. Their openness, their answers to my many questions and their knowledge of quality, health, safety and environment practices helped me understand the real needs behind this project.

I am also grateful to my teachers and to the members of the jury, for the time they devote to reading and evaluating this work.

Finally, I thank my family and friends for their constant support and encouragement throughout my studies.

Mohamed Sahbi Ben Kalaia
Academic Year 2025/2026

Abstract

Organisations that manage quality, health, safety and environment (QHSE) keep many documents that must agree with each other. Finding where these documents contradict each other is still done by hand, which is slow and error-prone, and comparing every passage of one document with every passage of another quickly becomes impractical. This study, carried out at Qualipro, reviewed the existing approaches to this problem, designed an architecture from that review, named COHERA, then built and evaluated a proof of concept. The central idea is to let a graph of the documents decide which parts are worth comparing, and to call on a language model only for the hard cases, always backed by exact quotes from the documents. On two test documents, the graph avoided 95% of the comparisons without losing a single inconsistency, and the system found 16 of the 19 inconsistencies they contain. These results are promising, but they were measured on a small test set.

Keywords: Document consistency, Dependency graph, QHSE, Natural language inference, Large language models, French normative documents.

Résumé

Les organisations qui pilotent la qualité, l'hygiène, la sécurité et l'environnement (QHSE) s'appuient sur de nombreux documents censés rester cohérents entre eux. Repérer leurs contradictions se fait encore à la main : le travail est long, et bien des écarts échappent à la relecture. Or confronter chaque passage d'un document à chaque passage d'un autre devient vite irréalisable. Menée chez Qualipro, cette étude a passé en revue les approches existantes, conçu à partir de cette revue une architecture baptisée COHERA, puis développé et évalué une preuve de concept. Son principe : un graphe des documents désigne les passages qui méritent d'être comparés, et un modèle de langage n'intervient que pour les cas difficiles, toujours en s'appuyant sur des citations exactes des documents. Sur deux documents de test, le graphe évite 95 % des comparaisons sans perdre une seule incohérence, et le système retrouve 16 des 19 incohérences qu'ils contiennent. Ces résultats sont encourageants, mais ils restent mesurés sur un jeu de test restreint.

Mots-clés : Cohérence documentaire, Graphe de dépendances, QHSE, Inférence en langage naturel, Grands modèles de langage, Documents normatifs.

Contents

Acknowledgements	ii
Abstract	iii
Résumé	iv
List of Abbreviations	x
1 Introduction	1
1.1 Context: a QHSE document set is a system, not a pile of files	1
1.2 Problem statement: a quadratic space with a poor signal-to-noise ratio	1
1.3 Objectives	2
1.4 Approach	2
1.5 Structure of the report	3
2 Internship Framework	4
2.1 The host organisation	4
2.2 The problem in the organisation’s terms	5
2.3 The subject and its deliverables	5
2.4 Methodology used	6
2.5 Work plan	6
2.6 A taxonomy of documentary inconsistencies	7
2.7 Working environment and technical stack	8
3 State of the Art	9
3.1 Formal verification with SMT solvers	9
3.2 Ontology-based consistency checking	11
3.3 Rule-based semantic extraction	12
3.4 Similarity-based candidate generation	13
3.5 Natural language inference	15
3.6 Large language models for document-level detection	16
3.7 Graph-enhanced language models	17
3.8 Synthesis: properties of a suitable architecture	18
4 Proposed Architecture	21
4.1 Design principles and overall pipeline	21
4.2 L0: Segmentation into self-contained normative units	22
4.3 L1: Hybrid extraction into Clause Frames	23
4.3.1 The Clause Frame	23
4.3.2 Rules first	24

4.3.3	The language model as a complement	25
4.4	L2: The dependency graph	25
4.4.1	Modelling principle and schema	25
4.4.2	The cross-document bridge	27
4.4.3	The condition algebra	28
4.4.4	The comparison key	29
4.4.5	The document hierarchy	29
4.5	L3: Targeting the candidate pairs	30
4.5.1	Five channels	30
4.5.2	Fusion, filters and budget	31
4.5.3	The metric that drives targeting	31
4.6	L4: The verification cascade	31
4.6.1	Stage A: symbolic detectors	32
4.6.2	The four-case scope rule	33
4.6.3	Stage B: bidirectional natural language inference	34
4.6.4	Stage C: a language model judge conditioned by the graph	34
4.7	L5: Consolidation and arbitration	35
4.7.1	From pair verdicts to findings	35
4.7.2	Criticality and hierarchical arbitration	35
4.8	L6: Reporting	35
4.9	Alternatives considered and rejected	36
4.10	Design risks and mitigations	36
4.11	Summary: six structuring decisions	36
5	Evaluation and Results	38
5.1	Experimental setup	38
5.2	Metrics	38
5.3	The graph built on the corpus	39
5.4	Targeting results	40
5.5	Detection results	40
5.6	Cost and reliability	41
5.7	Effect of the NLI stage	41
5.8	Undetected inconsistencies	42
5.9	Limitations	43
5.10	Chapter summary	43
6	Conclusion	44

List of Figures

1.1	The quadratic space of clause pairs and its signal-to-noise problem	2
2.1	Internship timeline	6
3.1	Formal verification of rules with an SMT solver	10
3.2	Ontology-based consistency checking	11
3.3	Conflict detection through rule-based semantic extraction	13
3.4	Two-phase detection: similarity-based candidates, then validation	14
3.5	Contradiction detection with a pre-trained NLI cross-encoder	15
3.6	Redact-and-retry with a constrained filter (RnR+CF)	17
3.7	GraphCheck: knowledge graphs encoded by a GNN and given to a frozen LLM	18
3.8	Properties a suitable architecture should combine	20
4.1	Overview of the proposed pipeline	22
4.2	The segmentation chain of layer L0	23
4.3	Anatomy of a Clause Frame	24
4.4	Schema of the dependency graph	27
4.5	The cross-document bridge	28
4.6	The condition algebra	29
4.7	The targeting layer	30
4.8	The verification cascade	32
4.9	The four-case scope rule as a decision tree	34
4.10	Anatomy of the judge’s prompt	34
4.11	The HTML report	36
5.1	Measured flow from the pair space to the findings (remote profile)	40

List of Tables

2.1	Host organisation fact sheet	5
2.2	Taxonomy of inconsistencies between QHSE documents	7
2.3	Technical stack of the proof of concept	8
3.1	Strengths and weaknesses of formal verification with SMT solvers	10
3.2	Strengths and weaknesses of ontology-based consistency checking	12
3.3	Strengths and weaknesses of rule-based semantic extraction	13
3.4	Strengths and weaknesses of similarity-based candidate generation	15
3.5	Strengths and weaknesses of natural language inference	16
3.6	Strengths and weaknesses of document-level LLM detection	17
3.7	Strengths and weaknesses of graph-enhanced language models	18
3.8	Qualitative comparison of the reviewed families	19
4.1	The seven layers of the architecture	21
4.2	Fields of the Clause Frame	23
4.3	Rule-based extraction methods	24
4.4	Examples of dimensional normalisation	25
4.5	Quantity registry: role, dimension and monotonicity	25
4.6	Node labels of the graph	26
4.7	Edge types of the graph	26
4.8	Alias resolution cascade	28
4.9	Two clauses with the same comparison key and different values	29
4.10	The documentary hierarchy	29
4.11	The five targeting channels	30
4.12	Expected reduction for two documents of about 300 clauses (design estimate)	31
4.13	From inconsistency type to detector and stage	32
4.14	Force and polarity of the deontic modalities	33
4.15	Deontic force gap and verdict of detector A1	33
4.16	The four-case scope rule	33
4.17	Output contract of the judge	34
4.18	Factors of the criticality score	35
5.1	Corpus and ground truth	38
5.2	Run conditions	38
5.3	Evaluation metrics	39
5.4	Nodes of the graph built on the corpus	39
5.5	Relationships of the graph built on the corpus	39
5.6	Targeting results	40
5.7	Flow of the 72 candidate pairs through the cascade	40

5.8	Detection results per profile	41
5.9	Cost and reliability of the judge	41
5.10	Effect of stage B on the remote profile	41
5.11	Inconsistencies missed by the remote profile	42
5.12	Detectors specified but not implemented	42
5.13	Limitations of the evaluation	43

List of Abbreviations

API	Application Programming Interface
ATEX	Atmosphères Explosibles (explosive atmospheres)
CLI	Command-Line Interface
CPU	Central Processing Unit
EDM	Electronic Document Management
EPI / PPE	Équipement de Protection Individuelle / Personal Protective Equipment
F1	F1-score
FMEA	Failure Mode and Effects Analysis
FP	False Positive
GNN	Graph Neural Network
GPU	Graphics Processing Unit
HSE	Health, Safety, Environment
HTML	HyperText Markup Language
IDF	Inverse Document Frequency
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
k-NN	k-Nearest Neighbours
LLM	Large Language Model
NER	Named Entity Recognition
NLI	Natural Language Inference
OWL	Web Ontology Language
PoC	Proof of Concept
POS	Part-of-Speech
QHSE	Quality, Health, Safety, Environment
QSE	Quality, Safety, Environment
RACI	Responsible, Accountable, Consulted, Informed
RAG	Retrieval-Augmented Generation
RRF	Reciprocal Rank Fusion
SBVR	Semantics of Business Vocabulary and Business Rules
SHACL	Shapes Constraint Language
SI	Système International (International System of Units)
SMT	Satisfiability Modulo Theories
TP	True Positive
YAML	YAML Ain't Markup Language

Chapter 1

Introduction

1.1 Context: a QHSE document set is a system, not a pile of files

A Quality, Health, Safety and Environment (QHSE) document set is the collection of procedures, policies, instructions and records by which an organisation manages its compliance with safety, environmental and quality requirements.

A single industrial site typically holds dozens of such documents, and they are not independent: they form a hierarchical pyramid in which a policy is declined into a manual, a manual into procedures, and procedures into instructions and records. Each lower level is expected to implement, and never loosen, the requirements of the level above it.

These documents are written and revised by different people, at different times, and often by different departments. Each revision can introduce a discrepancy with another document: a different deadline, a different periodicity, a responsibility assigned to two different roles, or a reference to a standard that has since been withdrawn.

A contradiction between two documents of the same pyramid is not a harmless typo. If a procedure requires a quarterly inspection while the policy it declines only recommends one, an operator facing an audit can legitimately invoke the more permissive document, and the stricter requirement becomes unenforceable in practice.

In the QHSE domain, where documentary compliance is examined during audits and certification, such inconsistencies are a direct operational and regulatory risk. Today they are usually found by reading, which does not scale with the size of the document set.

1.2 Problem statement: a quadratic space with a poor signal-to-noise ratio

If two documents contain n_1 and n_2 normative clauses, the number of clause pairs that could in principle contradict each other is $n_1 \times n_2$. For two documents of around 300 clauses each, this already gives roughly 90 000 pairs, and the number grows quadratically with the size of the document set.

Cost is not the central difficulty; the real difficulty is signal-to-noise. Among 90 000 candidate pairs, only a handful are genuine contradictions. A detector with an apparently good precision of 95% would still raise on the order of 4 500 false alarms, which is a report nobody would read to the end.

The question is therefore not only whether a model can recognise a contradiction between two

sentences, but which pairs, out of all possible ones, are worth examining at all, and how to examine them at a cost that remains compatible with local, confidential processing of internal documents.

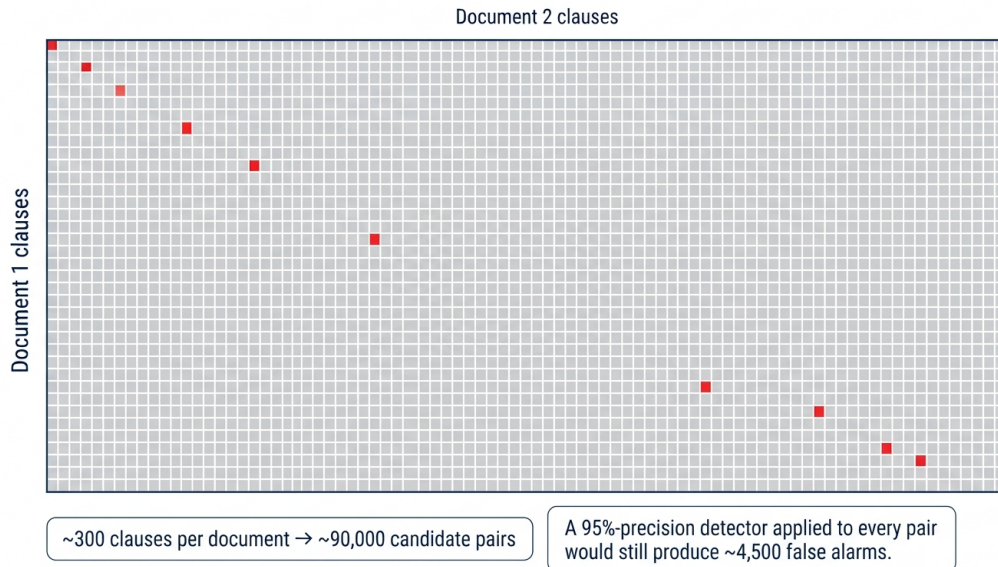


Figure 1.1: The quadratic space of clause pairs and its signal-to-noise problem

1.3 Objectives

The internship subject sets four objectives:

1. Analyse and formalise the problem, by producing a taxonomy of documentary inconsistencies such as divergent values, broken references and contradictory requirements.
2. Review the state of the art and benchmark the existing approaches, including natural language inference, graph representations of documents, requirement traceability and commercial tools.
3. Propose and compare candidate architectures, making explicit for each one the processing pipeline, the granularity of the graph and an estimate of its cost.
4. Recommend a solution and validate its central mechanism with a targeted proof of concept on a small set of annotated documents.

The subject frames the internship as an upstream study rather than product development: the proof of concept is meant to demonstrate the feasibility of a key mechanism, not to be a complete prototype.

1.4 Approach

The working hypothesis of the subject is that modelling documents as a dependency graph, whose nodes are requirements and whose edges are explicit references and semantic links, makes it possible to focus costly verifications on a small number of candidate pairs instead of comparing everything with everything. This report follows that hypothesis.

1.5 Structure of the report

- Chapter 2 presents the host organisation, Qualipro, and the framework of the internship.
- Chapter 3 reviews the families of approaches most relevant to detecting documentary inconsistencies and identifies what a suitable architecture should retain from them.
- Chapter 4 describes the proposed design in detail.
- Chapter 5 reports what was measured on the proof of concept, including where the results fell short of what was expected.
- Chapter 6 summarises what was demonstrated, what was not, and what a follow-up internship should take up next.

Chapter 2

Internship Framework

2.1 The host organisation

The internship took place at Qualipro, within its Innovation and R&D team. Qualipro is a software solution for Quality, Health, Safety and Environment (QHSE) management, designed to digitalise an organisation's integrated management system: quality, occupational health and safety, environment and food safety [1]. It is published by Saphir Consult, a consulting firm founded by Hichem B'Chir, whose career in quality management began as a factory director before he founded Saphir Consult and designed the Qualipro software [1].

Footprint. According to the publisher, Qualipro draws on 25 years of experience and is used in more than 40 countries by more than 1 500 customers, with over 600 partners [1]. The company lists offices in France (Lyon), Tunisia (Menzah 4) and Morocco (Casablanca) [1].

Qualipro is one of the software offerings of the Imagine Human group, which provides digital solutions to improve companies' performance in health, safety, environment, quality and quality of life at work [1].

Product. Qualipro is modular. Its modules cover, among others, electronic document management (EDM), corrective and preventive action plans, audits, risks and opportunities, performance indicators, suppliers and customers, non-conformities, metrology, environment, safety, food safety, failure mode and effects analysis (FMEA), legal compliance and change control.

The solution is aligned with the requirements of ISO 9001:2015, ISO 22000, ISO 14001:2015, ISO 45001, IATF 16949 and ISO 13485, and targets sectors with strict compliance requirements such as the automotive, aerospace, pharmaceutical, healthcare and food industries [1].

Relevance to this internship. Qualipro's customers use it to hold and maintain exactly the kind of document set described in Section 1.1: procedures, policies and instructions subject to audit and certification. Its document management module is where such documents live and are revised, and therefore where the question studied here arises: how to detect inconsistencies between documents automatically, at a controlled cost. Table 2.1 summarises the organisation.

Table 2.1: Host organisation fact sheet

Item	Qualipro
Activity	Publisher of QHSE management software
Publisher	Saphir Consult (founder: Hichem B'Chir)
Parent group	Imagine Human
Offices	Lyon (France), Menzah 4 (Tunisia), Casablanca (Morocco)
Footprint	More than 40 countries, more than 1 500 customers, over 600 partners
Standards covered	ISO 9001:2015, ISO 22000, ISO 14001:2015, ISO 45001, IATF 16949, ISO 13485
Host team	Innovation and R&D

2.2 The problem in the organisation's terms

Seen from a QHSE team, the problem is operational rather than mathematical. A quality manager who revises a policy needs to know which procedures derived from it are affected by the revision, before the new version is approved. An auditor who finds two documents in disagreement does not only ask whether they differ, but which of the two is at fault.

Answering this second question requires knowing the position of each document in the hierarchy: a lower-level document may be stricter than the document it derives from, which is a legitimate specialisation, but it may not be more permissive.

2.3 The subject and its deliverables

The internship subject, registered as “*Étude d’architecture pour l’analyse de cohérence documentaire par graphe de dépendances*” [EN: Architecture study for documentary consistency analysis through a dependency graph], is an upstream research and scoping study conducted over two months within Qualipro’s Innovation and R&D team.

It does not aim at developing a product or at integration into an existing one, but at producing a rigorous analysis and an argued architecture recommendation, validated by a proof of concept of its central mechanism, that can serve as the basis for a later implementation phase.

The work compares two documents with each other and looks for the inconsistencies between them. It produced three main deliverables, presented in this report:

- a **taxonomy of documentary inconsistencies** adapted to the QHSE domain, presented in Section 2.6;
- a **state of the art** of the approaches able to detect such inconsistencies, each assessed in terms of strengths and weaknesses (Chapter 3);
- a **candidate architecture**, designed from the conclusions of the state of the art, and a **proof of concept** that implements it on an annotated corpus (Chapters 4 and 5).

A single candidate architecture was retained. Rather than building several complete alternatives, the state of the art was used to identify the properties a suitable architecture should combine, and one architecture was designed to combine them. The hypotheses and limitations

of this choice are discussed with the results.

2.4 Methodology used

The study proceeded in four steps:

1. define the taxonomy (Section 2.6), so that the rest of the work could be measured against explicit types of inconsistencies;
2. review the state of the art;
3. design the candidate architecture from the conclusions of the review;
4. implement the architecture as a proof of concept and evaluate it.

No real QHSE document set was available for the evaluation. The corpus was therefore built for the purpose: two QHSE documents were written by the author, a procedure and the safety policy it derives from, and contradictions were then injected into them. The resulting corpus contains 19 annotated inconsistencies, which serve as the reference against which the proof of concept is evaluated.

Building the corpus this way has a direct consequence for the interpretation of the results. A system evaluated only on injected inconsistencies tends to overestimate its performance, since injected cases are usually cleaner than the ones found in real documents. The figures reported in Chapter 5 are therefore feasibility indications, not performance that generalises to any document set.

2.5 Work plan

The internship ran over nine weeks, from July to August 2026. The work moved from understanding the problem and studying existing approaches, to designing and building the system, and finally to evaluating it and writing this report (Figure 2.1).

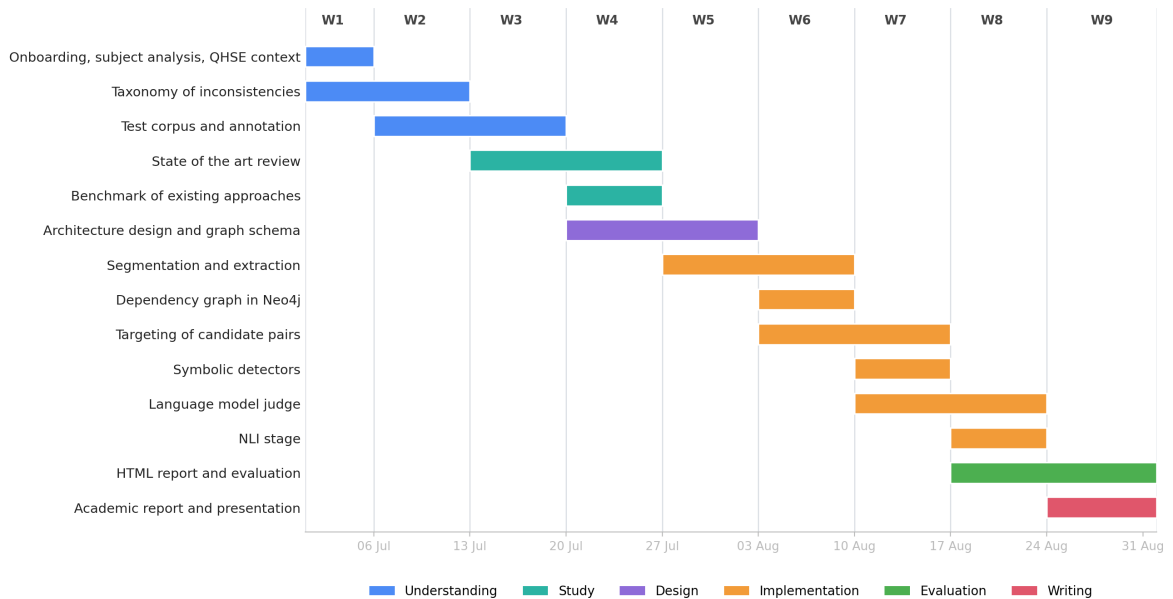


Figure 2.1: Internship timeline

2.6 A taxonomy of documentary inconsistencies

The first step of the study was to define what counts as an inconsistency between two QHSE documents. The taxonomy starts from the eight types of self-contradiction defined by ContraDoc [2] (Negation, Numeric, Content, Relation, Factual, Causal, Perspective and Emotion) and translates each of them into its QHSE meaning.

It then adds three types that are specific to a documentary system: Temporal, Derogation and Hierarchical. Each type is also given a priority, reflecting how serious and how frequent it is expected to be in a QHSE document set. Table 2.2 summarises the eleven types.

Table 2.2: Taxonomy of inconsistencies between QHSE documents

Origin	Type	QHSE meaning	Priority
ContraDoc	Negation	Obligation versus prohibition on the same action	High
	Numeric	Divergent values, thresholds, deadlines or periodicities	High
	Content	A divergent attribute: actor, place or object	High
	Relation	Mutually exclusive responsibilities	Medium
	Factual	Broken reference, obsolete or divergent standard	Medium
	Causal	The trigger does not match the effect	Medium
	Perspective	Divergent prescriptive strength	Medium
	Emotion	Almost absent from normative documents	Not covered
QHSE-specific	Temporal	Contradictory clauses in force at the same time, or an expired clause still cited	Medium
	Derogation	Undeclared derogation, or one that targets a missing or repealed document	Medium
	Hierarchical	A lower-level document more permissive than the one it derives from	High

The following examples illustrate each covered type with a pair of statements taken from two different documents.

- **Negation.** One document states that wearing a helmet is mandatory in zone A; the other states that it is not required in zone A.
- **Numeric.** A quarterly inspection in one document and a half-yearly inspection in the other; or a deadline of 5 working days against a deadline of 48 hours.
- **Content.** Approval by the QSE Manager in one document, and by the workshop manager in the other.
- **Relation.** Two documents assign the responsibility for carrying out the same task to two different roles.
- **Factual.** One document cites ISO 45001:2018 while the other still cites OHSAS 18001, a withdrawn standard; or a document refers to a section that does not exist.
- **Causal.** An immediate shutdown in case of an incident in one document, against a shutdown only after approval in the other.
- **Perspective.** Wearing gloves is recommended in one document and compulsory in the other.
- **Temporal.** A clause applicable from 1 January 2025 is contradicted by a clause that

was repealed on 31 December 2024 but is still cited.

- **Derogation.** A document declares a derogation from another document that has been repealed.
- **Hierarchical.** An instruction requires an annual inspection, while the procedure it derives from requires a quarterly one.

The three QHSE-specific types do not appear in generic taxonomies of textual contradictions, because they are not properties of the text itself but of the documentary system: the versions of a document, the derogations it declares, and its position in the document hierarchy. They are nevertheless among the most frequent findings in a real QHSE audit.

2.7 Working environment and technical stack

The proof of concept was developed in Python. Table 2.3 lists the main components. Two execution profiles are available for the language model, with the same pipeline. The remote profile calls a larger model through an API and is the reference profile of the evaluation. The local profile keeps all data on the machine: it is the confidential alternative, since QHSE documents are often confidential.

Table 2.3: Technical stack of the proof of concept

Role	Component
Language	Python
Segmentation and linguistic analysis	spaCy, <code>fr_core_news_lg</code>
Sentence embeddings	BAAI/bge-m3
Natural language inference	<code>cmarkea/distilcamembert-base-nli</code>
Graph database	Neo4j Community Edition
Local language model	<code>saiga_mistral_7b_gguf</code> , served by LM Studio
Remote language model	<code>llama-3.3-70b-versatile</code> , through Groq
Schema validation	pydantic
Command-line interface	typer
HTML report	jinja2
Tests	pytest

Chapter 3

State of the Art

Detecting conflicts between statements is an old problem that has been attacked from very different directions, from formal logic to large language models [3]. This chapter reviews the families of approaches most relevant here, that is, methods that decide whether two normative or factual statements are incompatible. They are presented from the most symbolic to the most neural, ending with hybrid approaches that combine a graph with a language model.

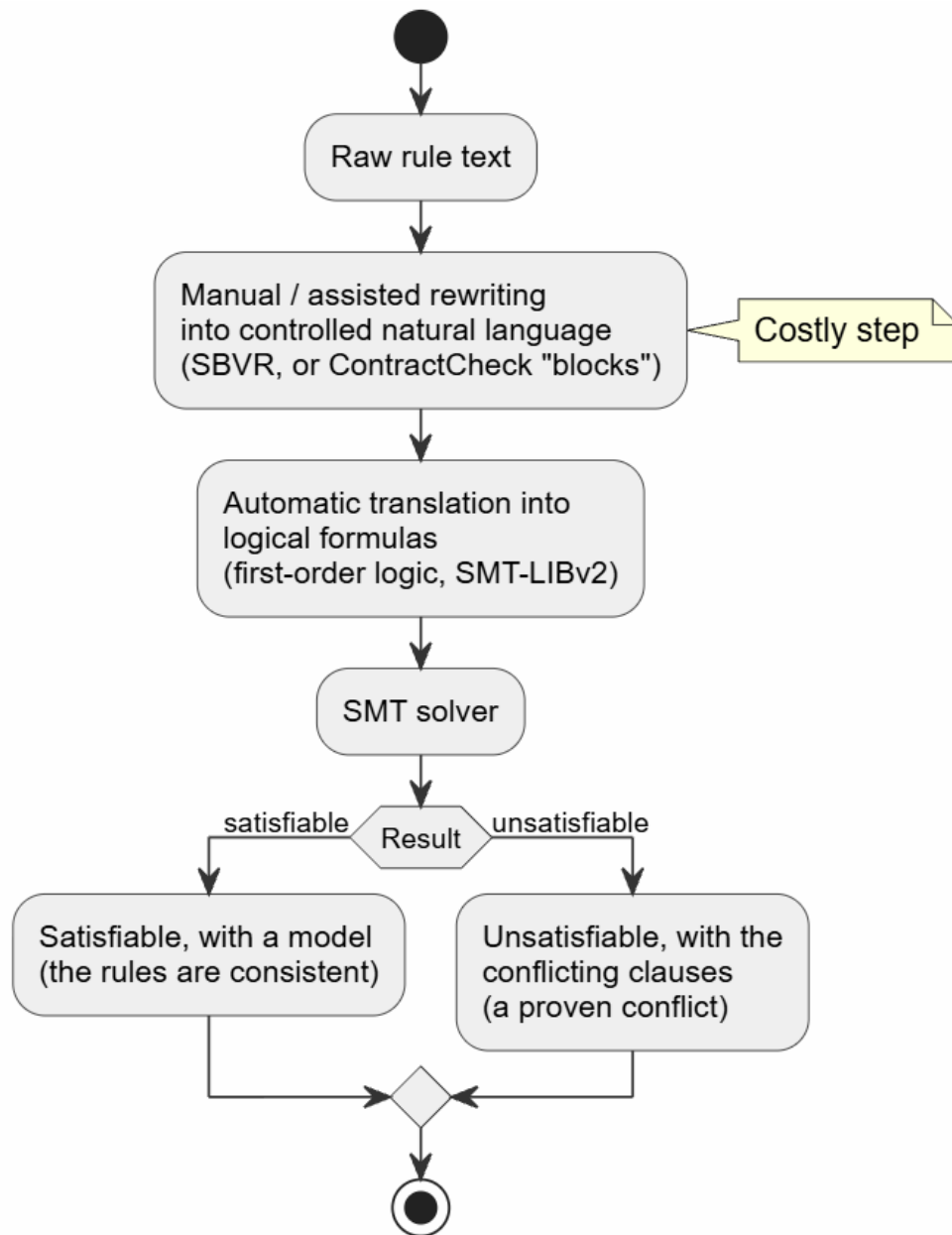
Each family is described, illustrated by a diagram and assessed in a table of strengths and weaknesses. A synthesis of the properties a suitable architecture should combine closes the chapter.

3.1 Formal verification with SMT solvers

Methodology. This family translates rules into logical formulas and asks a Satisfiability Modulo Theories (SMT) solver, such as Z3 [4], whether they can all hold at the same time. The rules must first be written in a controlled natural language, that is, a restricted subset of natural language with an unambiguous logical reading. The Semantics of Business Vocabulary and Business Rules (SBVR) standard [5] is the usual choice for business rules, because it is grounded in first-order logic.

Mitra et al. [6] detect anomalies in SBVR business rules in two steps. A directed graph built over the rules first reveals structural anomalies; the rules are then modelled in SMT-LIBv2 and a solver detects a second set of anomalies. The anomalies covered are conflicting rules (the same premise leading to mutually exclusive conclusions), redundancies (rules that can never fire, rules subsumed by others, logically equivalent rules) and circularity.

ContractCheck, proposed by Khoja et al. [7] for share purchase agreements, follows the same principle. The contract text is first encoded in a structured natural language format (“blocks”) defined by a domain ontology, and the blocks are translated into first-order logic assertions. An SMT solver then either returns a model showing that the contract can be executed or proves the existence of conflicting clauses. Figure 3.1 summarises the pipeline.

**Figure 3.1:** Formal verification of rules with an SMT solver**Table 3.1:** Strengths and weaknesses of formal verification with SMT solvers

Strengths	Weaknesses
• Verdicts are proven, not estimated: the solver exhibits a model showing that the rules can hold together, or identifies the conflicting clauses. • Detects conflicts involving chains of several rules, not only pairs. • Cheap, deterministic execution once the rules are encoded.	• The text must first be rewritten in a controlled language, so the cost moves from reasoning to formalisation. • Each document type needs its own vocabulary or ontology, as in ContractCheck. • Anything not encoded is invisible to the solver; raw French QHSE text cannot be given to it directly.

3.2 Ontology-based consistency checking

Methodology. In this family, a domain expert models the concepts of the domain and the constraints between them as an ontology, typically in the Web Ontology Language OWL 2 [8]. The documents are then represented as instance data of this ontology, and a description logic reasoner such as HermiT [9] checks whether the whole is consistent and explains the axioms responsible for any inconsistency.

For regulatory texts specifically, Heimsbakk and Torkelsen [10] argue that the open world assumption of OWL, together with limitations in its expressiveness, makes it less suitable for modelling rules and regulations. Under the open world assumption, a missing fact is unknown rather than false, whereas compliance checking needs to conclude that a required element is absent.

They instead model regulatory requirements as constraints in the Shapes Constraint Language (SHACL), which has closed world semantics, and let a SHACL engine validate instance data against them. They also report that the resulting model could be maintained by domain experts without prior knowledge of ontology management. Figure 3.2 summarises the approach.

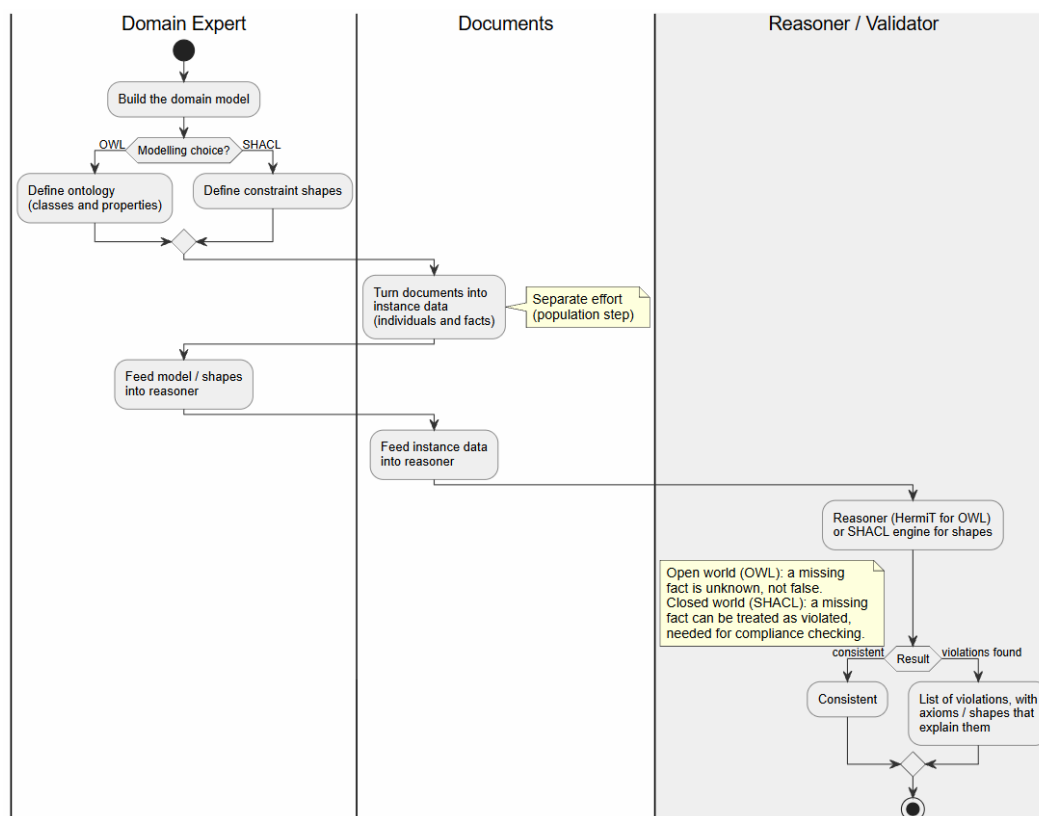


Figure 3.2: Ontology-based consistency checking

Table 3.2: Strengths and weaknesses of ontology-based consistency checking

Strengths	Weaknesses
• An explicit, shared domain model, in which actors, actions, objects and conditions become first-class elements. • Standard, mature reasoners that explain why a set of facts is inconsistent. • Once built, the model is reusable for every document of the same domain.	• Costly modelling, requiring both domain and ontology expertise. • The open world assumption of OWL fits regulations poorly; compliance checking needs a closed world language such as SHACL. • Documents must still be turned into instance data, and anything outside the modelled scope remains invisible.

3.3 Rule-based semantic extraction

Methodology. Instead of asking the author to write in a controlled language, this family extracts a structured representation from ordinary text with linguistic rules, and then compares the structured representations.

FSARC, proposed by Guo et al. [11] for software functional requirements, is representative. It defines a semantic meta-model of a requirement as an eight-tuple of semantic elements, annotates these elements automatically from the linguistic features of each requirement, and detects seven types of conflicts with heuristic rules based on text patterns and semantic dependencies. It needs no specific requirement template and no external resource.

On four requirement datasets, the authors report a recall of nearly 100% and an average precision of 83.88%. Figure 3.3 summarises the pipeline.

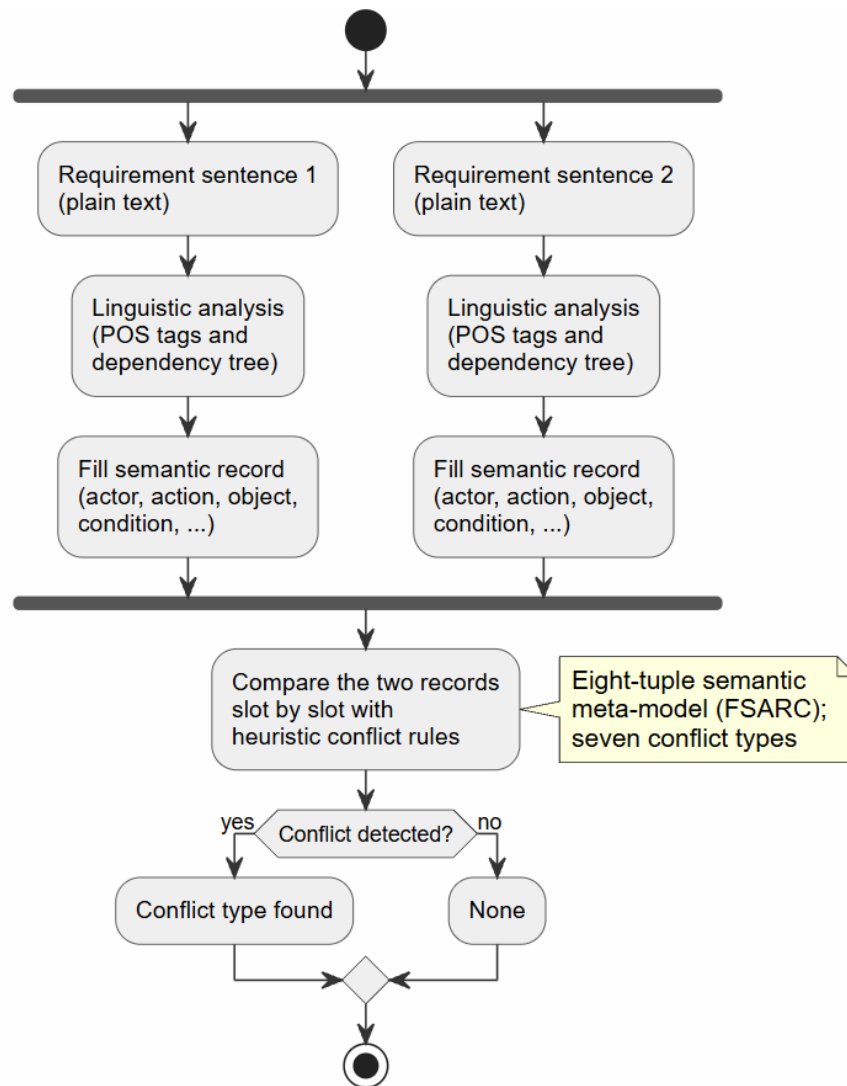


Figure 3.3: Conflict detection through rule-based semantic extraction

Table 3.3: Strengths and weaknesses of rule-based semantic extraction

Strengths	Weaknesses
• Works on ordinary text and imposes no controlled language on the author. • Once statements are structured, many conflicts reduce to comparing fields (two actors, two values), which is cheap and easy to explain. • No training data is needed.	• The heuristics are tied to one language and one genre of text, here English software requirements, and must be rewritten for another. • Extraction errors propagate directly into missed or false conflicts. • Designed for a single set of requirements, with no notion of a document hierarchy in which one document declines another.

3.4 Similarity-based candidate generation

Methodology. Comparing every statement with every other one is quadratic. This family therefore splits detection into two phases: a cheap similarity measure first selects the pairs that are worth examining, and a second step validates them.

Similarity is usually computed with sentence embeddings, that is, dense vectors that represent

the meaning of a sentence. A bi-encoder such as Sentence-BERT [12] encodes each sentence independently, so the similarity of two sentences is a simple cosine between vectors and the nearest neighbours of a sentence can be retrieved from an index without scoring every pair. Multilingual models such as BGE-M3 [13] make this available for French.

S3CDA, proposed by Malik et al. [14], applies this two-phase design to requirements. It first identifies potentially conflicting requirement pairs using semantic similarity, then validates them by analysing the domain-specific entities they share. Evaluated on five real-world datasets, it consistently outperforms large language models (GPT-4o, Llama 3, Claude Sonnet 3.5, Gemini 1.5) in domain-specific settings, while the authors note that the language models remain promising on general datasets. Figure 3.4 summarises the approach.

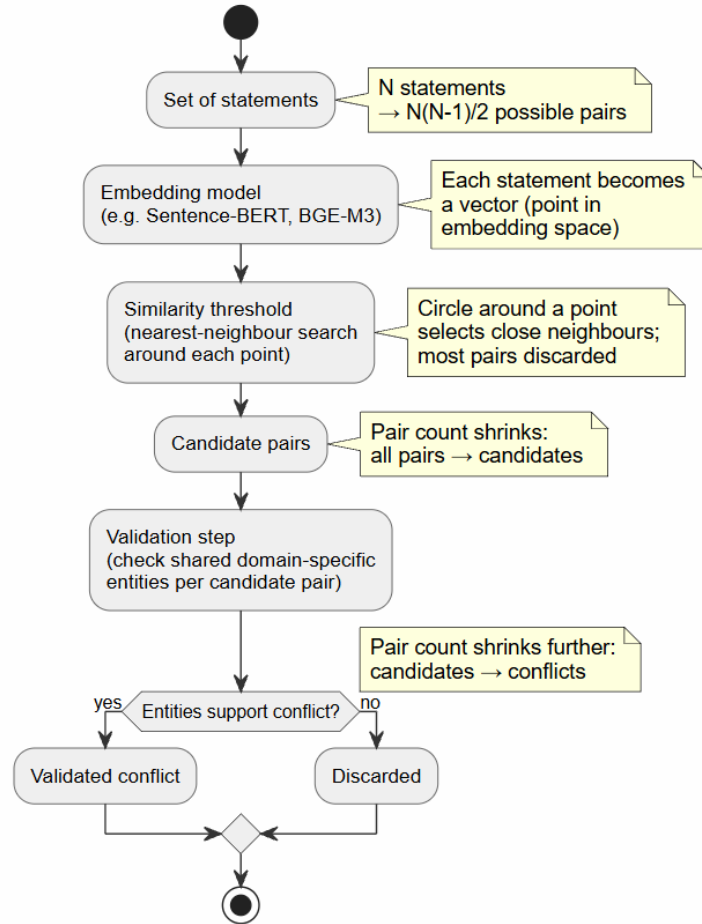


Figure 3.4: Two-phase detection: similarity-based candidates, then validation

Table 3.4: Strengths and weaknesses of similarity-based candidate generation

Strengths	Weaknesses
• No longer quadratic: only a small set of candidate pairs reaches the validation step. • Embeddings are cheap to compute and can be indexed. • A domain-aware validation step performs well against general language models on specialised data.	• Similarity measures proximity, not contradiction: two contradictory clauses and two coherent clauses can be equally similar. • Pairs that conflict while using different vocabulary can fall below the threshold and are then lost for good. • Shared entities show that two statements talk about the same thing, not that they disagree.

3.5 Natural language inference

Methodology. Natural language inference (NLI) is the task of deciding whether a hypothesis sentence is entailed by, contradicts, or is neutral with respect to a premise sentence. De Marneffe et al. [15] framed contradiction detection as a task of its own within this setting.

Large annotated corpora, SNLI [16] and MultiNLI [17], made it possible to train neural classifiers for it, and XNLI [18] extended the evaluation to other languages, including French.

For French, pre-trained encoders such as CamemBERT [19], and its lighter distilled version DistilCamemBERT [20], can be fine-tuned on NLI data. The resulting model is a cross-encoder: both sentences are read together, and the model outputs a probability for each of the three labels. Figure 3.5 summarises the approach.

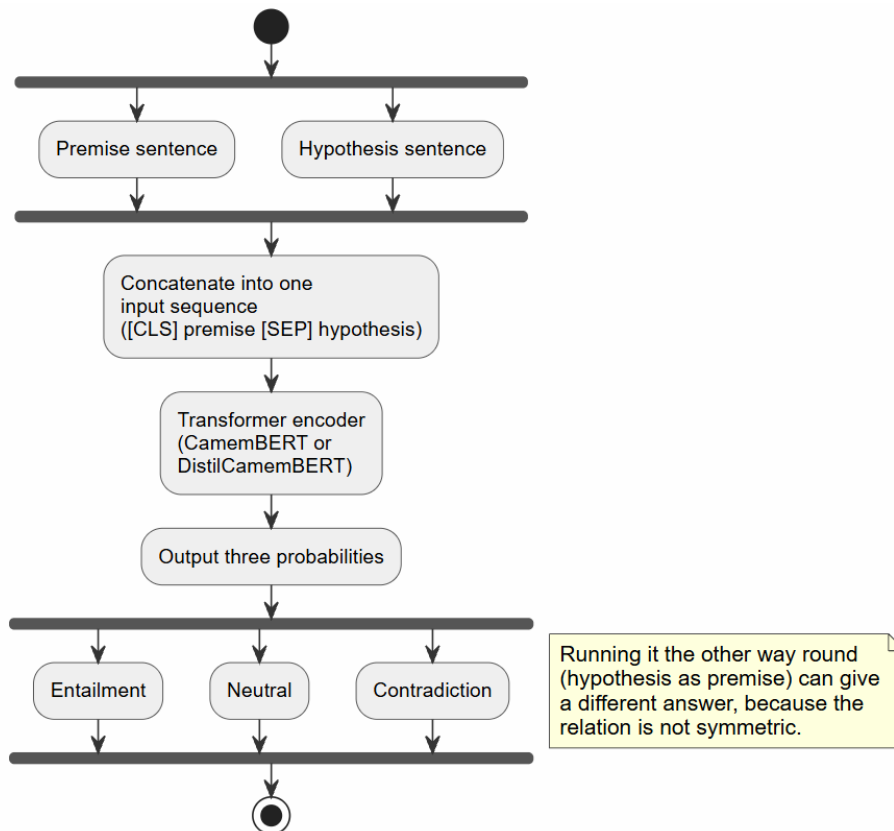
**Figure 3.5:** Contradiction detection with a pre-trained NLI cross-encoder

Table 3.5: Strengths and weaknesses of natural language inference

Strengths	Weaknesses
• Mature and available off the shelf for French, with no training required from the user. • Much cheaper per pair than a large language model. • A graded contradiction score that can be thresholded or used to rank pairs.	• Trained on generic, short sentences rather than on French normative text. • Returns a probability, but no evidence and no explanation. • Applied to every pair, it inherits the quadratic cost and floods the result with false alarms.

3.6 Large language models for document-level detection

Methodology. Large language models (LLMs) can be asked directly whether a document contradicts itself, and where. ContraDoc [2] is the reference dataset for this setting. It contains 449 self-contradictory and 442 non-contradictory documents from three domains (news, stories and Wikipedia), ranging from 100 to 2 200 tokens, built by having an LLM insert contradictions that human annotators then verified and tagged.

Each contradiction is tagged with one or more of eight types: Negation, Numeric, Content, Perspective, Emotion, Relation, Factual and Causal. Evaluating GPT-3.5, GPT-4, PaLM2 and LLaMAv2, the authors find that GPT-4 performs best and can outperform humans, but remains unreliable on contradictions that require more nuance and context.

Tan et al. [21] focus on the evidence the model returns. Their redact-and-retry framework asks the LLM whether the document is inconsistent and which sentences show it; as long as the answer is yes, those sentences are removed from the document and the model is asked again, which lets it find further inconsistencies. A final filter call then re-analyses the collected sentences and keeps only those it considers truly inconsistent.

In the constrained variant (RnR+CF), which the authors find to be the best all-round approach, the filter must return at least one sentence. Because the model does not always reproduce the input sentences exactly, the authors match its output to the document approximately, with a TF-IDF cosine similarity of at least 0.8. Figure 3.6 summarises the framework.

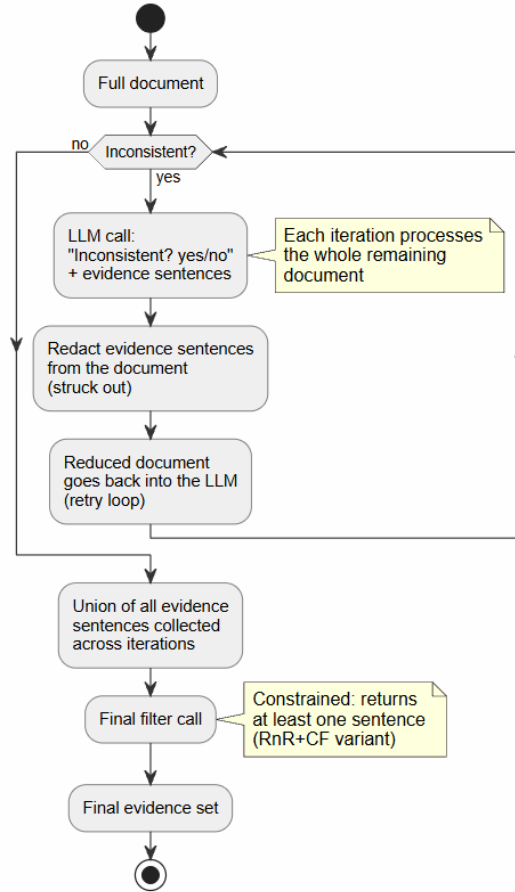


Figure 3.6: Redact-and-retry with a constrained filter (RnR+CF)

Table 3.6: Strengths and weaknesses of document-level LLM detection

Strengths	Weaknesses
• No training and no formalisation: the model reads the raw document. • Covers all contradiction types, including causal ones that rules cannot capture. • Returns evidence sentences rather than a bare yes or no.	• Every call processes the whole document, which requires a long context and therefore a large model. • Several calls per document, and even the strongest models remain unreliable on subtle cases. • The evidence is matched approximately, so it is not guaranteed to be a verbatim quotation of the document.

3.7 Graph-enhanced language models

Methodology. The last family represents the text as a knowledge graph and gives that graph to a language model. GraphRAG, proposed by Edge et al. [22], uses an LLM to derive an entity knowledge graph from the source documents and to pre-generate summaries of communities of closely related entities. At query time, each community summary produces a partial answer, and the partial answers are summarised into a final one. GraphRAG targets global questions over a whole corpus, where standard retrieval fails, rather than contradiction detection.

GraphCheck, proposed by Chen et al. [23], applies the idea to fact-checking a claim against a

document. An LLM extracts entity and relation triples from both, which form two knowledge graphs. A graph neural network (GNN) encodes each graph, a projector maps the graph embeddings into the embedding space of the verifier LLM, and the frozen LLM receives both the text and the graph embeddings and answers “supported” or “unsupported” in a single call.

The GNN is trained on about 14 000 synthetic samples, on four NVIDIA A100 80 GB GPUs. With Llama 3.3 70B as the verifier, GraphCheck reaches an overall balanced accuracy of 71.1% across seven benchmarks. Two findings of the paper matter here:

- the quality of the extracted graph drives the result: triples extracted by a small model missed key details and led to wrong predictions;
- simply writing the graph into the prompt as text brought only a marginal improvement over the base model.

Figure 3.7 summarises GraphCheck.

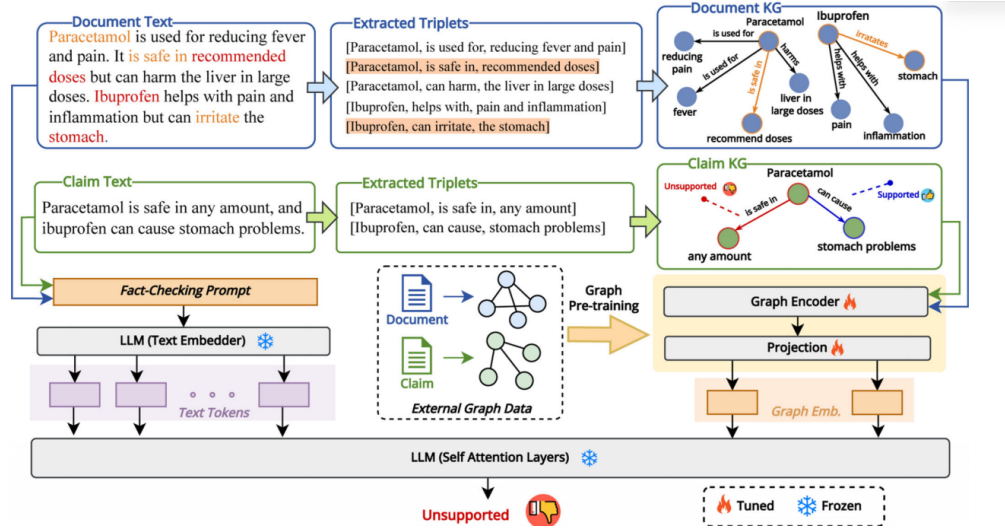


Figure 3.7: GraphCheck: knowledge graphs encoded by a GNN and given to a frozen LLM

Table 3.7: Strengths and weaknesses of graph-enhanced language models

Strengths	Weaknesses
• The graph captures relations spread across the text, including multi-hop chains. • A compressed, structured view of the document. • Verification takes a single call instead of one call per atomic fact.	• Training the GNN requires thousands of examples and data-centre GPUs. • Verifies one claim against one document, with a binary verdict and no localised evidence. • Results depend on the quality of the extracted graph. • Writing the graph into the prompt as text brings little.

3.8 Synthesis: properties of a suitable architecture

Table 3.8 compares the seven families on the criteria that matter for this project. The ratings are a qualitative assessment drawn from the descriptions above.

Two groups appear. Formal verification, ontologies and GraphCheck pay their cost up front, in formalisation, modelling or training. NLI applied to every pair and document-level LLM detection pay it at run time, in inference calls. No single family is cheap on both sides, gives verbatim evidence, and reads raw French normative text across several documents.

Table 3.8: Qualitative comparison of the reviewed families

Family	Upfront	Run time	Evidence	Raw text	Pair selection
SMT verification	High	Low	Proven conflict	No	Not needed
Ontologies	High	Low	Explained violation	No	Not needed
Rule-based extraction	Medium	Low	Structured fields	Yes	No
Similarity candidates	Low	Low	Shared entities	Yes	Yes
NLI	Low	Medium	None	Yes	No
LLM, document level	Low	High	Approximate	Yes	No
GraphCheck	High	Medium	None (binary)	Yes	No

A suitable architecture should therefore not choose one family, but combine the strongest property of several of them while avoiding the part that makes each one costly:

- **From formal verification:** some conflicts are a matter of logic rather than interpretation. An obligation and a prohibition on the same action cannot both hold, and this can be decided by a fixed rule, without any learned model.
- **From ontologies:** an explicit model of what a requirement is made of (who must do what, on which object and under which conditions), without paying for a full ontology and reasoner.
- **From rule-based extraction:** each statement is turned into a structured record before anything is compared, so that many inconsistencies reduce to comparing two fields, such as two values of the same deadline.
- **From similarity-based candidate generation:** a two-phase design, in which a cheap step selects the pairs worth examining and a more expensive step verifies them. Similarity alone measures proximity, however, and can miss pairs that are worded differently.
- **From natural language inference:** a cheap, pre-trained model that scores the contradiction between two sentences, used on selected pairs rather than on every pair.
- **From document-level LLM detection:** a language model for the cases that rules cannot handle, required to return evidence for its verdict, but without whole-document prompts.
- **From graph-enhanced approaches:** a graph as a structured, compressed representation of the documents.

GraphCheck’s finding that a graph written into a prompt helps little suggests that the main value of the graph lies less in making a model more accurate than in deciding which pairs deserve to be examined at all. Figure 3.8 summarises these properties; Chapter 4 presents an architecture designed to combine them.

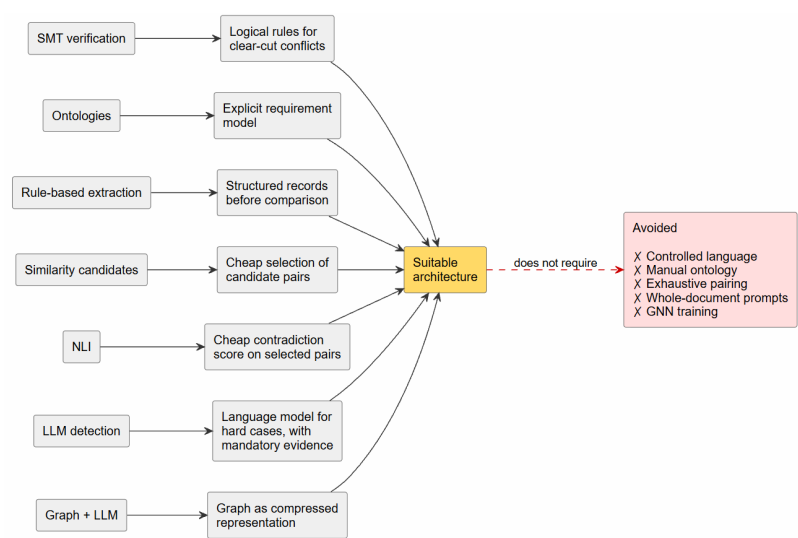


Figure 3.8: Properties a suitable architecture should combine

Chapter 4

Proposed Architecture

This chapter presents COHERA, the architecture designed to combine the properties identified in Chapter 3. Chapter 5 reports what the proof of concept implements and measures.

4.1 Design principles and overall pipeline

In plain words. The system first builds a graph of the documents, uses it to pick the few pairs of clauses worth checking, and checks each pair with the cheapest tool able to decide it.

The design rests on one idea:

The graph is not a way to store the documents. It is the mechanism that decides which pairs deserve to be checked, by which detector, and which ones the document set has already resolved by itself.

Four invariants follow:

- I1 Nothing expensive runs before targeting:** no model call on a pair the graph did not select.
- I2 The cheapest sufficient detector decides:** a value divergence is a comparison of two numbers, not a model call.
- I3 No verdict without evidence:** two exact quotations, the detector and the graph path.
- I4 The document set declares itself:** derogations, repeals, versions and the hierarchy are read before anything is reported.

Table 4.1 and Figure 4.1 summarise the seven layers.

Table 4.1: The seven layers of the architecture

Layer	Role	Output
L0 Segmentation	Cut the text into self-contained normative units	Clauses with offsets
L1 Hybrid extraction	Structure each clause, rules first	Clause Frames
L2 Graph construction	Link clauses, concepts, values, conditions	Dependency graph
L3 Targeting	Select the pairs worth checking	Candidate pairs
L4 Verification cascade	Check each pair, cheapest detector first	Pair verdicts
L5 Consolidation	Group verdicts, score them, find the faulty clause	Findings
L6 Reporting	Present the results	JSON and HTML reports

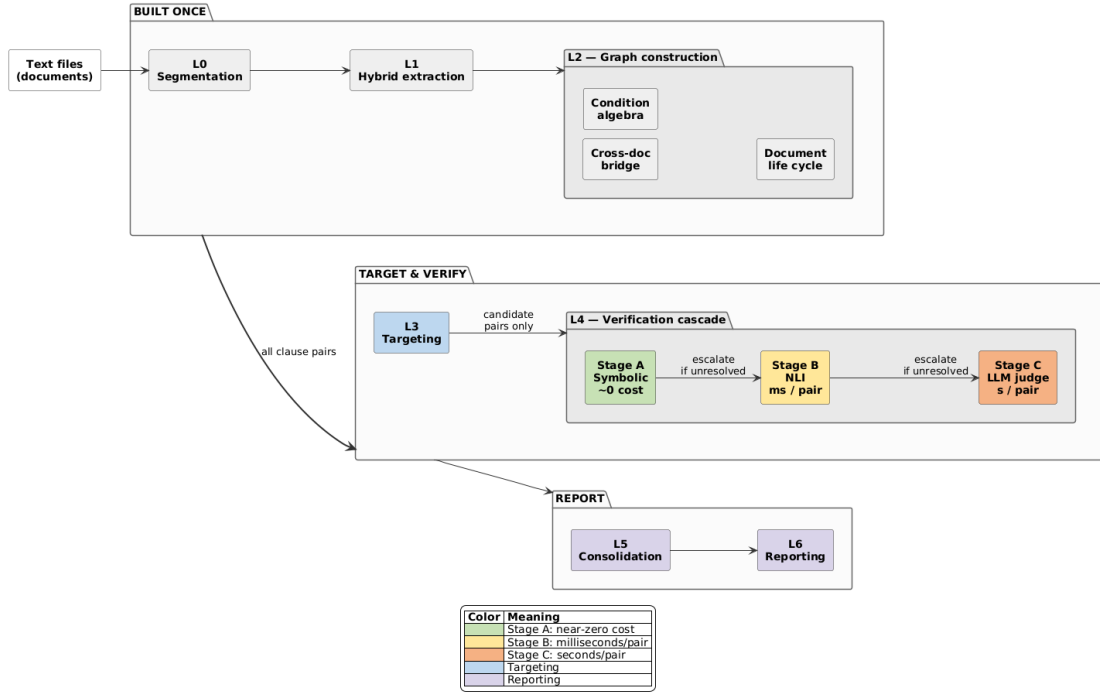


Figure 4.1: Overview of the proposed pipeline

4.2 L0: Segmentation into self-contained normative units

In plain words. The text is cut into self-contained pieces, each stating a single requirement.

A Clause is one atomic normative statement: a paragraph is too coarse, and a subject, predicate and object triple loses the modality and conditions that create QHSE conflicts. Seven steps produce the clauses (Figure 4.2):

1. **Normalisation** of characters and decimal separators.
2. **Structure detection**, with a flat fallback.
3. **List recomposition**: the lead-in (“The QSE Manager must:”) is copied onto each item.
4. **Tabular block detection**: each row becomes a clause.
5. **Sentence segmentation** with spaCy `fr_core_news_lg`.
6. **Qualification**: only normative sentences become clauses.
7. **Decontextualisation**: “It must be renewed every year.” becomes “The PPE inspection must be renewed every year.”

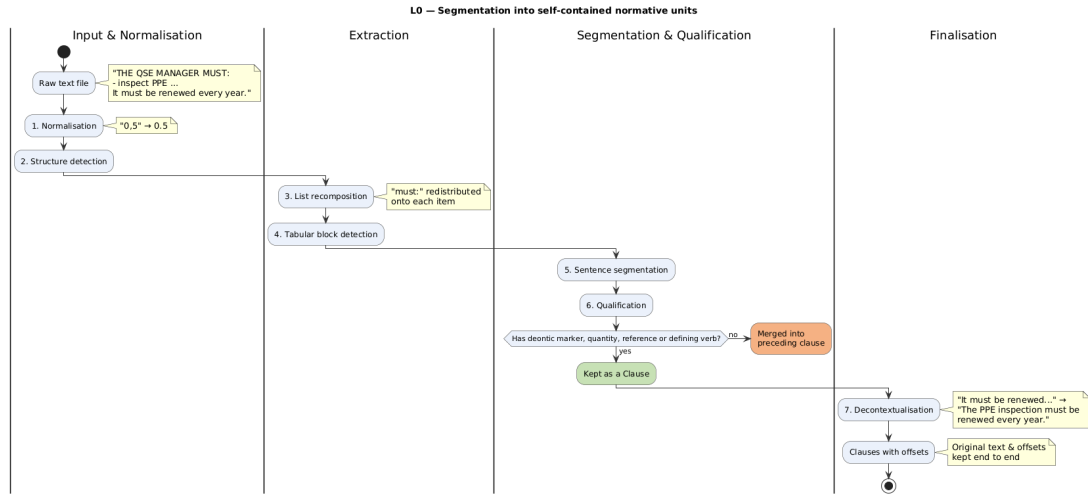


Figure 4.2: The segmentation chain of layer L0

Each clause keeps its original text and offsets, quoted as evidence, and a hash used by the incremental mode.

4.3 L1: Hybrid extraction into Clause Frames

In plain words. Each clause becomes a filled form: who must do what, on which object, how strongly, with which value and under which conditions.

4.3.1 The Clause Frame

Table 4.2 lists the fields of the frame and Figure 4.3 shows an example.

Table 4.2: Fields of the Clause Frame

Field	Content	Example
Statement type	Prescription, definition, observation, reference or derogation	Prescription
Modality and force	Obligation, prohibition, recommendation, permission	Obligation (force 3)
Negation	Whether the modality is negated	Yes: “must not validate”
Actor	Who bears the requirement, with a RACI role	QSE Manager (R)
Action	Lemmatised verb	Renew
Object	What the action applies to	PPE inspection
Quantities	Role, value, unit, dimension, SI value, operator	Periodicity, 3 months
Conditions	Typed conditions of applicability	In zone A (spatial)
Perimeter	Site, workshop, activity or population	Radès site
Scope quantifier	Universal or existential	Universal
Validity	Start and end dates	From 1 January 2024
References	Internal, standard or regulatory	ISO 45001:2018
Derogation	Declared derogation, if any	By derogation from PR-QSE-04 §3.1
Controlled risks	Hazards the clause controls	Fall from height
Provenance	Origin of each field: rule or LLM	Modality: rule

Anatomy of a Clause Frame

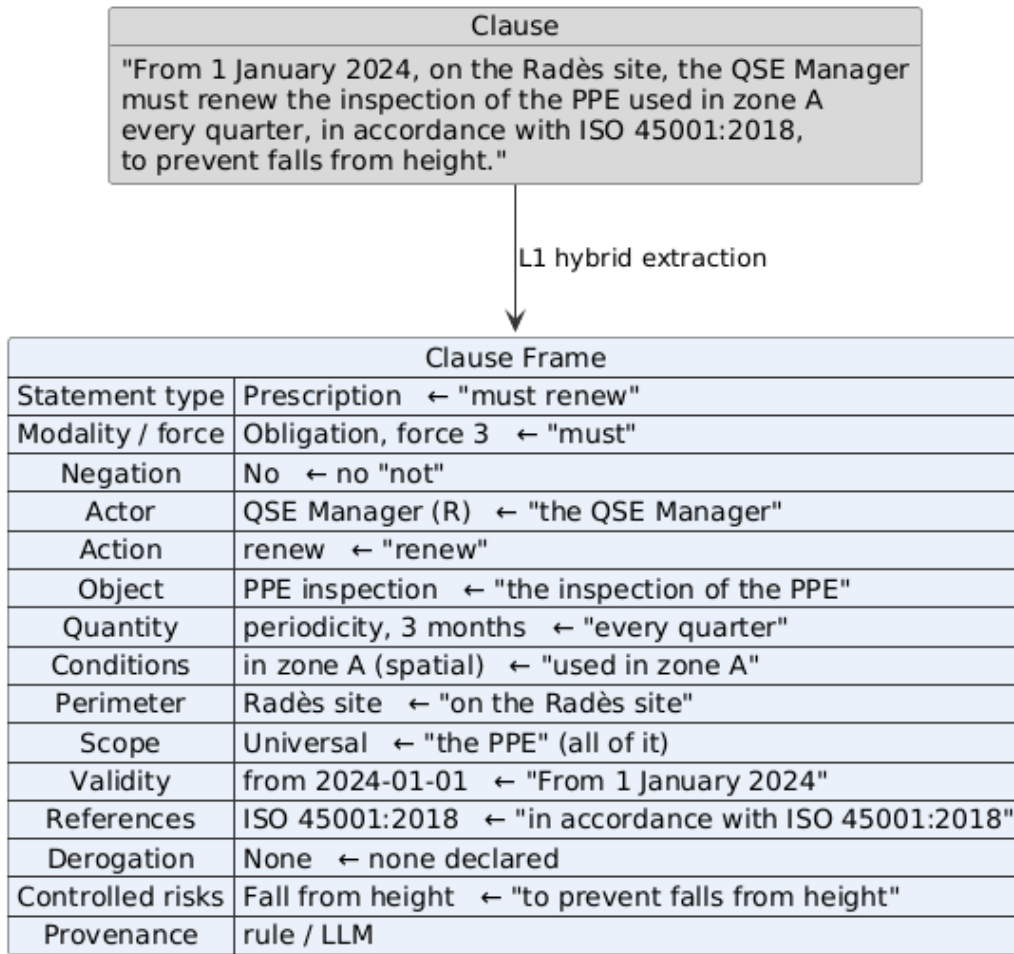


Figure 4.3: Anatomy of a Clause Frame

4.3.2 Rules first

Every field is first filled by rules on the French text (Table 4.3).

Table 4.3: Rule-based extraction methods

Field	Method
Modality and force	Deontic lexicon and syntactic patterns (must, is forbidden, may, it is recommended)
Negation	Scope of negation read from the dependency tree
Quantities	Rule grammar followed by dimensional normalisation
Conditions	Typed markers (in case of, when, if, except, in zone, beyond)
Perimeter	Applicability markers (applies to, on the site of, for any activity of)
Validity	Dates and markers (from, applicable on, until, repealed on)
Derogation	Explicit markers (by derogation from, notwithstanding)
References	Regular expressions for sections, document codes, standards, articles
Actor and RACI role	Role gazetteer, named entity recognition, RACI markers
Controlled risks	Lexicon of QHSE hazards
Scope quantifier	Quantifiers (every, each, any, some, where applicable)

Quantities are converted to SI units (Table 4.4); two quantities are comparable only if their

dimension and role match.

Table 4.4: Examples of dimensional normalisation

Expression	Dimension	Normalised value
5 working days	Time	432 000 s, working-day qualifier kept
10 days	Time	864 000 s, calendar days
quarterly	Periodicity	7 884 000 s
every 3 months	Periodicity	7 884 000 s
twice a year	Periodicity	15 768 000 s
85 dB(A)	Sound pressure	85, A-weighting kept
beyond 2 m	Length	2 m, operator >

The quantity registry gives each role its *monotonicity*, the direction in which a value is stricter (Table 4.5). It separates a specialisation from a contradiction (Section 4.6.2).

Table 4.5: Quantity registry: role, dimension and monotonicity

Role	Dimension	Stricter means	QHSE example
Deadline	Time	Smaller	Validation within 48 h
Periodicity	Periodicity	Smaller	Quarterly inspection
Retention period	Time	Larger	Archiving for 5 years
Training duration	Time	Larger	14 h of training
Qualification validity	Time	Smaller	Refresher every 3 years
Exposure limit (maximum)	Varies	Smaller	85 dB(A)
Trigger threshold (minimum)	Varies	Larger	Harness beyond 2 m
Safety distance	Length	Larger	10 m from an ATEX zone
Minimum headcount	Headcount	Larger	2 first aiders per team
Tolerance	Dimensionless	Smaller	±5%

4.3.3 The language model as a complement

A language model fills only the fields the rules left empty, with constrained output, read-only pre-filled fields and closed vocabularies. A symbolic detector gives a firm verdict only if every field it compared came from a rule. Two profiles exist: a local one, where no data leaves the machine, and a hybrid one with a larger remote judge.

4.4 L2: The dependency graph

In plain words. Clauses, concepts, values and conditions become nodes of one graph, linked by references, shared meanings and document derivations.

4.4.1 Modelling principle and schema

The graph is stored in Neo4j Community Edition, with one label per kind of thing and sub-labels for domain specialisations (Tables 4.6 and 4.7, Figure 4.4). Quantities, conditions and findings are nodes, so that they can be traversed, compared once, and gathered.

Table 4.6: Node labels of the graph

Layer	Label	Role
Document	Document	Source file: type, hierarchy level, version, dates, status
	Section	Structural node
	Clause	Normative unit, with its frame properties and embedding
Semantic	Concept	Normalised domain term, with eight sub-labels
	Quantite	Reified quantity: dimension, role, SI value, operator
	Condition	Reified, typed condition of applicability
External	NormeExterne	Cited standard: code, version, status
	ExigenceExterne	Enforceable article of a regulation
Result	Constatation	Consolidated finding: type, severity, criticality, status
	Anomalie	Single-clause defect, such as a broken reference

Table 4.7: Edge types of the graph

Edge	From → To	Meaning
<i>Structure and extraction</i>		
CONTIENT	Document, Section → Section, Clause	Contains
SUIT	Clause → Clause	Reading order
MENTIONNE	Clause → Concept	Mentions
IMPOSE_A	Clause → Actor	Requirement bears on, with RACI role
PORTE	Clause → Quantite	Fixes this quantity
SOUS_CONDITION	Clause → Condition	Applies only under
S_APPLIQUE_A	Document, Clause → Perimeter	Applies to
MAITRISE	Clause → Risk	Controls this risk
EXIGE_COMPETENCE	Clause → Competence	Requires this qualification
DEFINIT	Clause → Concept	Defines
<i>References and external standards</i>		
RENVOIE_A	Clause → Clause, Section, Document	Explicit cross-reference
CITE_NORME	Clause → NormeExterne	Cites a standard
CITE_EXIGENCE	Clause → ExigenceExterne	Cites a regulatory article
CONTIENT_EXIGENCE	NormeExterne → ExigenceExterne	Structure of a standard
REPLACE	NormeExterne → NormeExterne	Supersedes
<i>Document life cycle</i>		
ANNULE_ET_REPLACE	Document → Document	Cancels and replaces
DECLINE	Document → Document	Declines (derives from)
DEROGE_A	Clause → Clause, Document	Declared derogation
REPLACE_CLAUSE	Clause → Clause	Same clause in two versions
<i>Cross-document alignment</i>		
ALIAS_DE	Concept → Concept	Same meaning
RECROISER	Condition → Condition	Can hold together
INCLUS_DANS	Condition → Condition	Is a special case of
DISJOINT_DE	Condition → Condition	Never hold together

Edge	From → To	Meaning
<i>Pipeline and results</i>		
PAIRE_CANDIDATE	Clause → Clause	Selected for checking
INCOHERENT_AVEC	Clause → Clause	Conflict verdict
SPECIALISE	Clause → Clause	Stricter restriction, compatible
IMPLIQUE	Constatacion → Clause	Clause involved in a finding
SIGNALE	Anomalie → Clause	Flags a single-clause defect
CONCERNE	Constatacion → Concept, NormeExterne	Business object of a finding

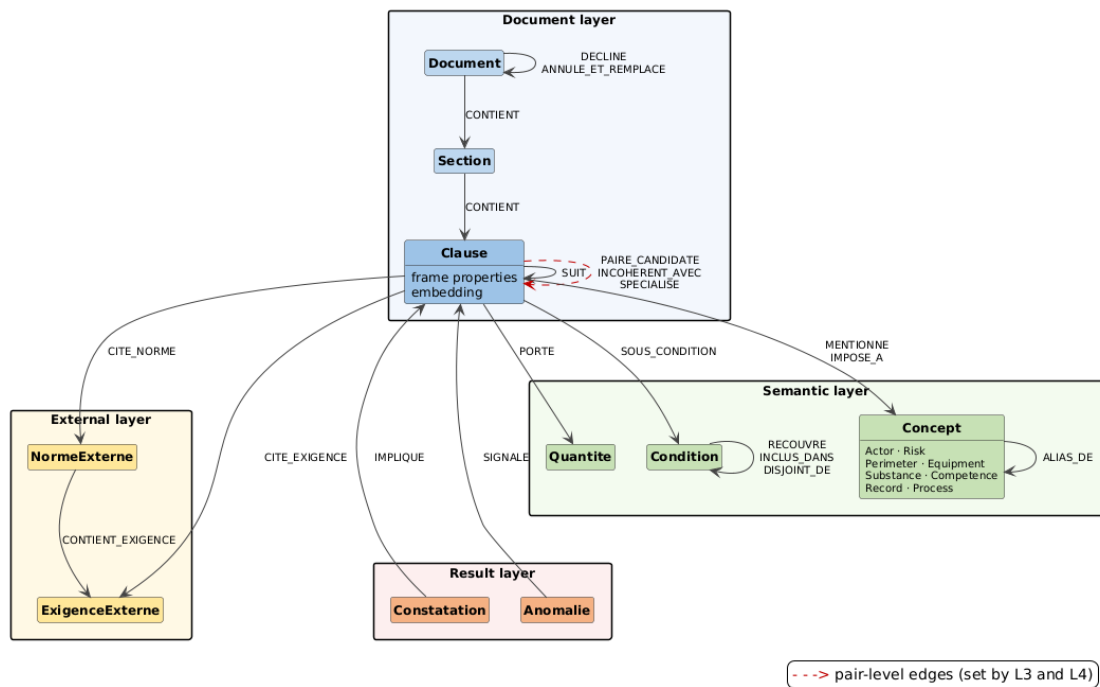


Figure 4.4: Schema of the dependency graph

4.4.2 The cross-document bridge

Two departments rarely use the same words: without alignment, “HSE Manager” and “Safety Officer” stay distinct and no candidate pair links the two documents (Figure 4.5). Aliases are resolved by a cascade of increasing cost (Table 4.8) and grouped under a canonical representative. A wrong alias makes unrelated clauses comparable, so aliases are traced, exposed as revisable hypotheses, and never enough alone for a firm verdict.

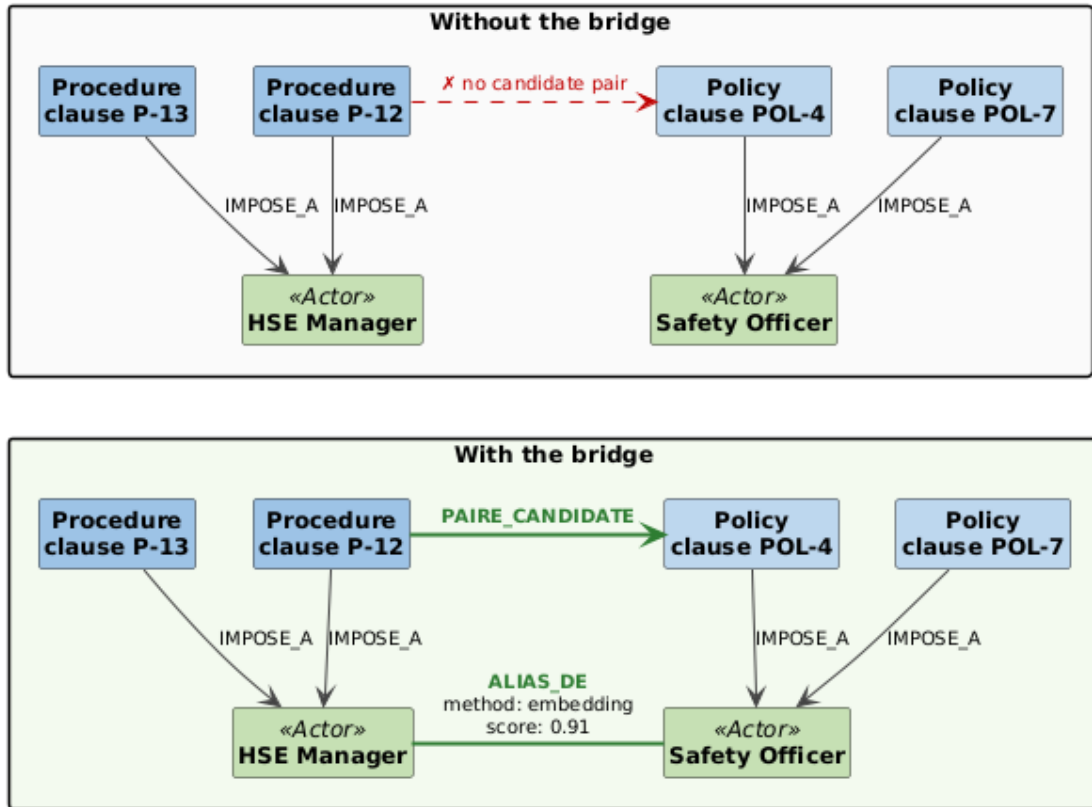


Figure 4.5: The cross-document bridge

Table 4.8: Alias resolution cascade

Step	Method	Decision	Recorded as
1	Normalised identity	Identical after normalisation	Exact, score 1.00
2	QHSE lexicon and acronyms	Listed equivalence	Lexicon, score 0.95
3	Vector similarity	Cosine ≥ 0.86 Cosine between 0.72 and 0.86 Cosine < 0.72	Vector, score = cosine Sent to step 4 Rejected
4	Language model arbitration	Yes, no or uncertain	LLM, score 0.8, or no alias

4.4.3 The condition algebra

Two different values conflict only if they apply to the same situation. Each distinct condition is compared with the others once, by typed rules first (place hierarchy, thresholds, dates), then vector similarity, then the language model on the grey zone. The result is stored as an overlap, inclusion or disjunction edge (Figure 4.6), reused by every clause pair.

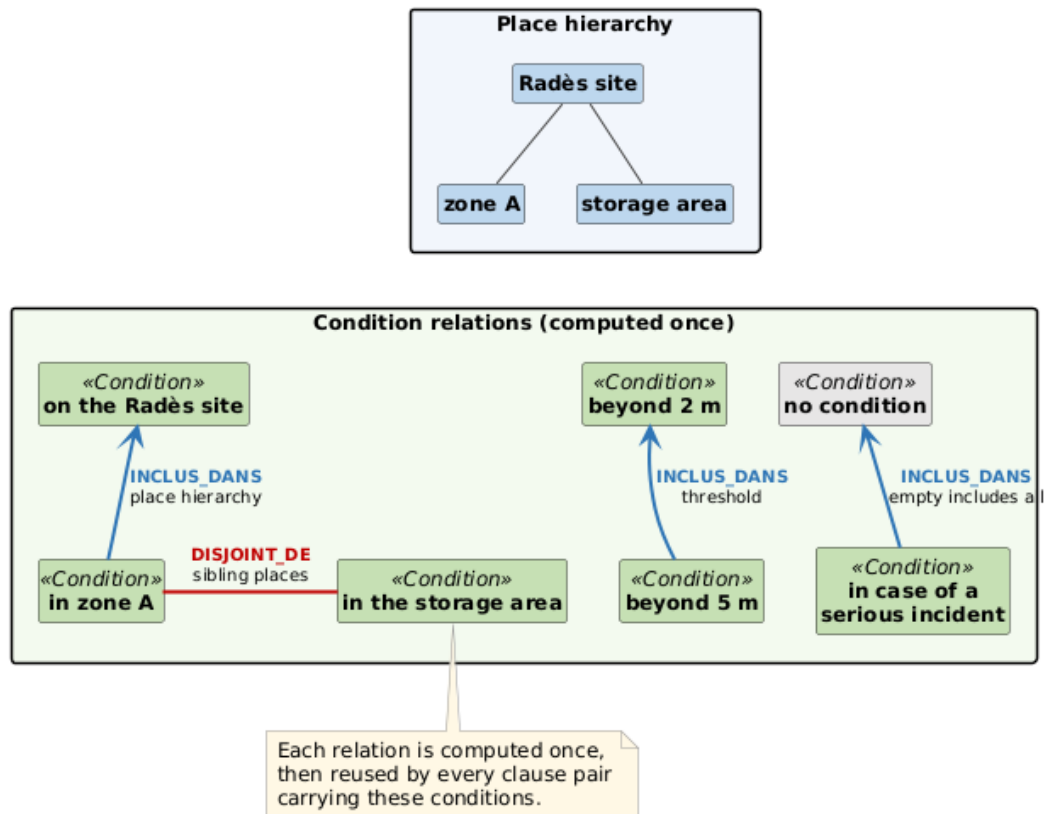


Figure 4.6: The condition algebra

4.4.4 The comparison key

The comparison key (canonical actor, action and object, plus the dimension and role of the quantity) makes two clauses with the same key and different values a near-certain divergence (Table 4.9).

Table 4.9: Two clauses with the same comparison key and different values

Clause	Actor	Action	Object	Dimension	Role	Value
Procedure	QSE Manager	validate	shutdown form	Time	Deadline	48 h (172 800 s)
Policy	QSE Manager	validate	shutdown form	Time	Deadline	5 days (432 000 s)

4.4.5 The document hierarchy

Each document has a level in the pyramid (Table 4.10), which carries the rule an auditor applies:

Table 4.10: The documentary hierarchy

Level	Documents
0	External requirements: labour code, decrees, standards
1	Policy, management commitment
2	QHSE manual, system documents
3	Procedures
4	Instructions, operating modes, work instructions
5	Records and forms

A lower-level document may be stricter than the document it derives from: this is a

legitimate declension. It may not be more permissive: this is a non-conformity. And no document, at any level, may be more permissive than an external requirement.

4.5 L3: Targeting the candidate pairs

In plain words. A few graph queries propose only the pairs of clauses that share something meaningful.

Targeting replaces the exhaustive $n_1 \times n_2$ comparison (Figure 4.7).

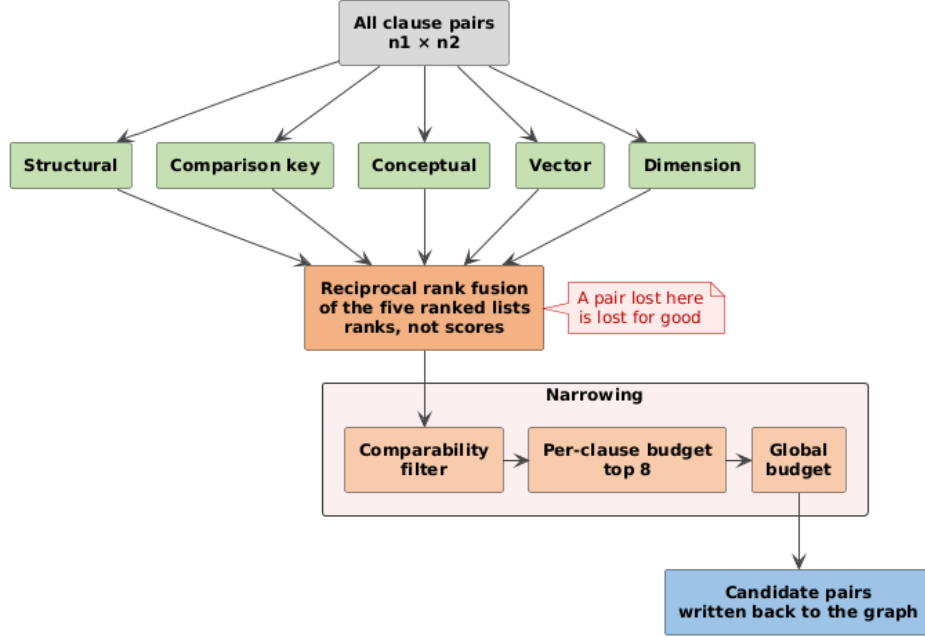


Figure 4.7: The targeting layer

4.5.1 Five channels

Five channels, each a graph query, propose pairs (Table 4.11).

Table 4.11: The five targeting channels

#	Channel	Pairs it proposes	Weight	Role
1	Structural	Linked by an explicit cross-reference	3.0	Very high precision
2	Comparison key	Same comparison key	3.0	Very high precision
3	Conceptual	At least 2 shared canonical concepts with IDF > 1.5	1.5	Main recall channel
4	Vector	Nearest neighbours ($k = 12$), cosine ≥ 0.70	1.0	Safety net
5	Dimension	Same quantity dimension and role, top 3 per clause	1.2	Vocabulary-independent

- **Conceptual:** because concepts are canonical, “HSE Manager” meets “Safety Officer” (Listing 4.1).
- **Vector:** a safety net over clause embeddings (BAAI/bge-m3).
- **Dimension:** no similarity threshold, so “within 24 hours” meets “within the week” although they share no word.


```

1 MATCH (a:Clause)-[ma:MENTIONNE]->(k:Concept)<-[mb:MENTIONNE]-(b:Clause)
2 WHERE a.doc_id < b.doc_id AND k.idf > 1.5
3 WITH a, b, sum(ma.poids * mb.poids * k.idf) AS score, count(k) AS partages
4 WHERE partages >= 2
5 RETURN a, b, score, 'CONCEPTUEL' AS canal

```

Listing 4.1: Conceptual targeting channel (Cypher, identifiers as in the design)

4.5.2 Fusion, filters and budget

Channel scores are not comparable, so pairs are merged by rank with Reciprocal Rank Fusion (RRF) [24]:

$$\text{RRF}(a, b) = \sum_{c \in \text{channels}} \frac{w_c}{60 + \text{rank}_c(a, b)} \quad (4.1)$$

A comparability filter and per-clause and global budgets then narrow the list, and the selected pairs are stored as candidate-pair edges.

4.5.3 The metric that drives targeting

The design expects about 1 200 candidates out of 90 000 for two 300-clause documents (Table 4.12). The pilot metric, however, is targeting recall (target 0.95 or more): a pair lost here is lost for good.

Table 4.12: Expected reduction for two documents of about 300 clauses (design estimate)

Step	Pairs
Theoretical space $n_1 \times n_2$	90 000
Union of the five channels	about 5 400
After the comparability filter	about 2 300
After per-clause and global budgets	about 1 200

4.6 L4: The verification cascade

In plain words. Each pair is checked by the cheapest tool able to decide it: rules, then a small contradiction model, then a language model that must quote the text.

Stage A applies symbolic detectors; pairs without a firm verdict go to stage B (NLI), and those it cannot settle to stage C, a language model judge (Figure 4.8, Table 4.13). The other five types of the taxonomy have no dedicated detector in the proof of concept; the design specifies one for each, left for future work.

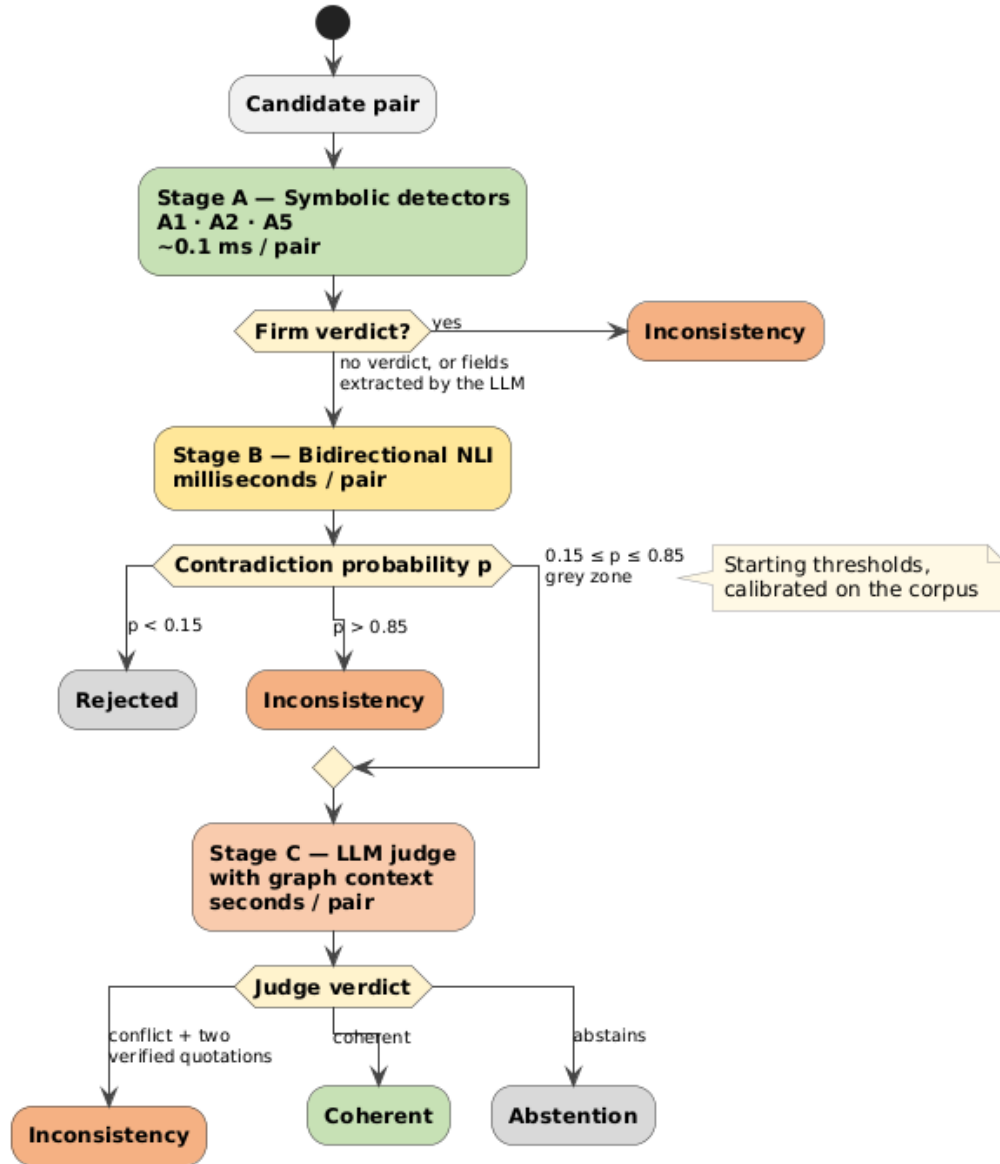


Figure 4.8: The verification cascade

Table 4.13: From inconsistency type to detector and stage

Type	Detector	Stage
Negation	A1, deontic conflict	A
Numeric	A2, value divergence	A
Factual	A5, references and standards	A
Causal	NLI, then LLM judge	B, then C
Perspective	A1, force gap	A, then B

4.6.1 Stage A: symbolic detectors

Stage A compares fields of the two Clause Frames directly, without any learned model. Three detectors are implemented:

- **A1, deontic conflict:** compares the modalities of two clauses that share the same actor, action and object.

- **A2, numeric divergence:** compares SI-converted values of the same dimension and role, under the scope rule of Section 4.6.2.
- **A5, references:** detects unresolved internal cross-references, and external standards that are obsolete or cited in divergent versions.

Detector A1 gives each modality a force, from the weakest to the strongest (Table 4.14). All modalities push towards performing the action, except the prohibition, which has the opposite polarity.

Table 4.14: Force and polarity of the deontic modalities

Modality	Typical marker	Force	Polarity
Permission	may (<i>peut</i>)	1	+
Recommendation	should (<i>il est recommandé</i>)	2	+
Obligation	must (<i>doit</i>)	3	+
Prohibition	must not (<i>il est interdit de</i>)	4	−

The verdict then depends on the gap between the two forces (Table 4.15). A gap of 1, such as *must* against *should*, is often a difference of emphasis rather than a contradiction, so it is not treated as firm and is escalated to stage B. Opposite polarity is checked first: an obligation and a prohibition on the same action cannot both be satisfied, so this case is a strong conflict even though their forces differ by only 1.

Table 4.15: Deontic force gap and verdict of detector A1

Situation	Verdict
Gap 0	No conflict
Gap 1	Perspective divergence, not firm, escalated
Gap 2 or more	Strong conflict
Opposite polarity (obligation against prohibition)	Strong conflict, whatever the gap

4.6.2 The four-case scope rule

A clause with no condition does not overlap every conditioned clause: 48 hours in general and 24 hours in case of a serious incident is a stricter special case, not a conflict. The four-case rule (Table 4.16, Figure 4.9) reads the scope relation from the condition algebra and “stricter” from the quantity registry.

Table 4.16: The four-case scope rule

Relation of the scopes	Restricted clause	Verdict
Identical	Different value	Contradiction
Disjoint	Any	No conflict
Inclusion (B within A)	Stricter than A	Specialisation, compatible
Inclusion (B within A)	More permissive than A	Contradiction
Indeterminate	Any	Escalation to stage C

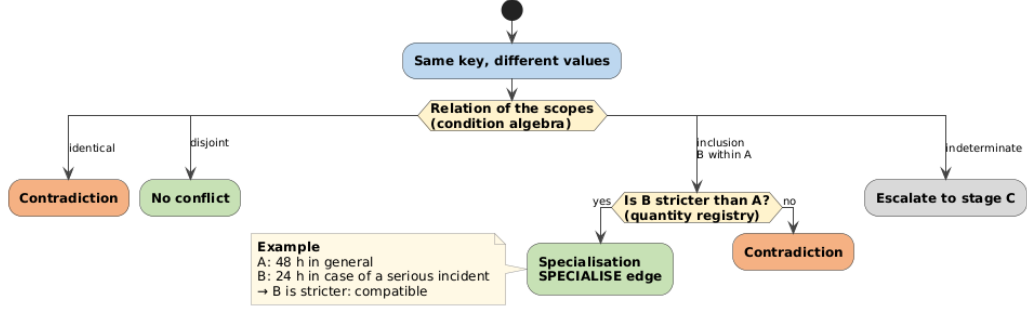


Figure 4.9: The four-case scope rule as a decision tree

4.6.3 Stage B: bidirectional natural language inference

Stage B uses the pre-trained NLI cross-encoder `cmrkea/distilcamembert-base-nli` (Section 3.5) in both directions, with thresholds calibrated on the corpus.

4.6.4 Stage C: a language model judge conditioned by the graph

The judge’s prompt gives both clauses with their frames and the facts the graph has established about the pair: aliases, scope relation, hierarchy and upstream signal (Figure 4.10). This is GraphCheck’s idea [23] without a trained GNN.

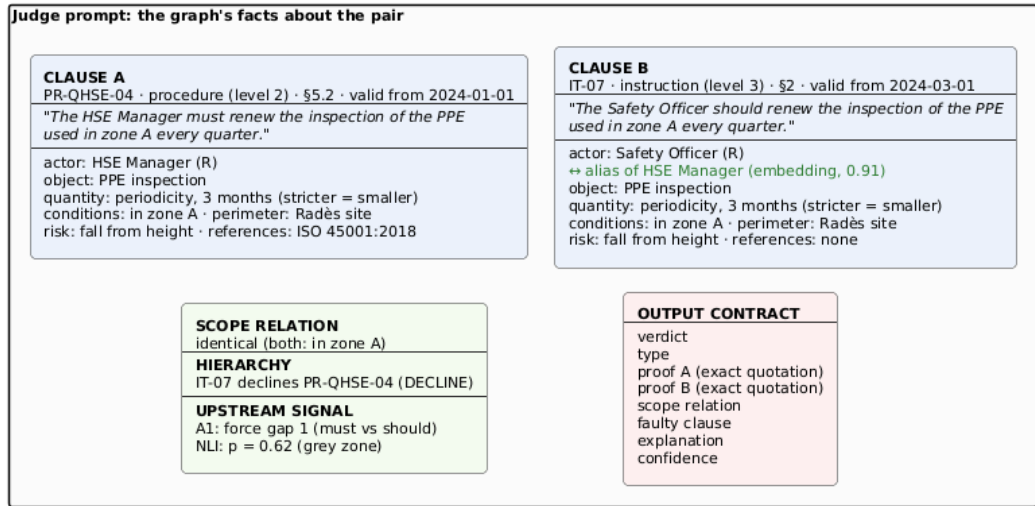


Figure 4.10: Anatomy of the judge’s prompt

The judge must return two exact proofs (Table 4.17), under five guardrails:

Table 4.17: Output contract of the judge

Field	Content
verdict	Inconsistency, coherent, specialisation, or undecidable
type	Type of the taxonomy
preuve_a, preuve_b	Exact excerpts of clause A and of clause B
relation_portees	Identical, inclusion, disjoint, or indeterminate
clause_fautive	A, B, or none
explication	Two sentences at most
confiance	Confidence between 0 and 1

1. **Constrained filter** [21]: proofs must be literal substrings, or the verdict is cancelled.

2. **Position bias:** severe pairs are judged in both orders.
3. **Self-consistency:** a two-out-of-three majority over three samples.
4. **Legitimate abstention**, sent to human review.
5. **Budget ceiling**, with unchecked pairs marked, never silently rejected.

Chapter 5 compares a local and a remote judge.

4.7 L5: Consolidation and arbitration

In plain words. Pair verdicts become findings: one per problem, scored by how serious it is, and naming the clause that should be fixed.

4.7.1 From pair verdicts to findings

The same problem often shows up on several pairs of clauses. Verdicts that share the same type, the same comparison key and the same conflicting values are merged into a single finding, so each problem is reported only once.

4.7.2 Criticality and hierarchical arbitration

Each finding gets a criticality score, which is the product of five factors (Table 4.18):

$$\text{criticality} = w_{\text{type}} \times w_{\text{severity}} \times w_{\text{confidence}} \times w_{\text{scope}} \times w_{\text{hierarchy}} \quad (4.2)$$

Table 4.18: Factors of the criticality score

Factor	What it measures
w_{type}	How serious this kind of inconsistency is
w_{severity}	How far apart the two values are, or how strong the conflict between the two rules is
$w_{\text{confidence}}$	How sure the system is about the verdict and about the extracted data
w_{scope}	How widely the rule applies. <i>Defined but not yet used: it is always 1.0 in the current version.</i>
$w_{\text{hierarchy}}$	2.0 when a clause contradicts an external standard, 1.5 when a lower-level document is more permissive than the one it comes from, 1.0 otherwise

The document hierarchy also tells which clause should be fixed: the more permissive clause of the lower-level document. When both documents are at the same level, the system does not decide and flags the finding for a human to review.

4.8 L6: Reporting

In plain words. The results come in three forms, all built from the same graph.

The proof of concept produces a JSON report for evaluation, an HTML report for auditors (Figure 4.11), with sections for alignment hypotheses, coherent pairs, derogations in force and uncovered zones, and the explorable graph. Every finding carries its two excerpts, its detector and its graph path.

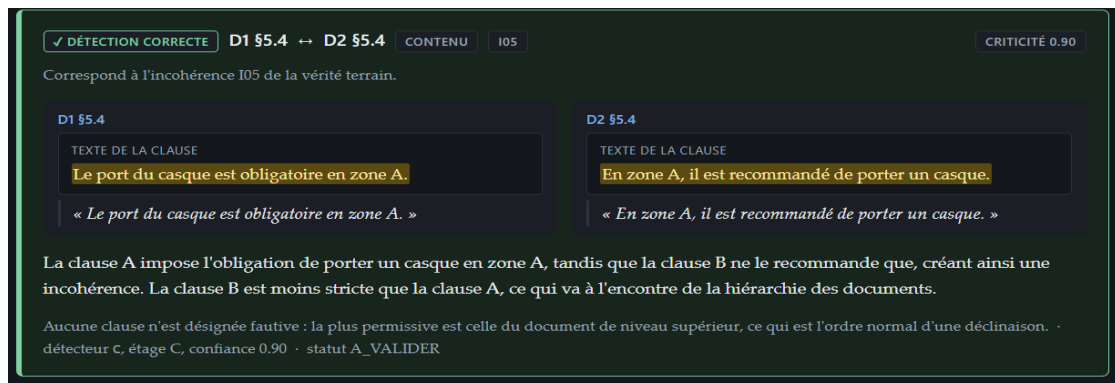


Figure 4.11: The HTML report

4.9 Alternatives considered and rejected

- **Pure vector retrieval:** proximity is not contradiction (Section 3.4).
- **OWL ontology and reasoner:** modelling cost; its deontic model is kept as graph nodes.
- **Controlled language and SMT solver:** formalisation bottleneck; its deontic table is kept in A1.
- **GraphCheck with a trained GNN:** training cost (Section 3.7); its idea is kept in the judge's prompt.
- **Supervised classifier:** no annotated French QHSE corpus exists.
- **RnR+CF on concatenated documents:** long context and coarse localisation.
- **A graph built entirely by a language model:** costly and unreliable frames.

4.10 Design risks and mitigations

- **Wrong aliases (main risk):** traced, revisable, never enough alone for a verdict.
- **Weak local extraction:** rules first, remote fallback.
- **Unstructured text:** heuristics with a flat fallback.
- **Low targeting recall:** five redundant channels.
- **Wrong scope comparison:** condition algebra and four-case rule.
- **NLI out of domain:** never the final judge.
- **Volatile free quotas:** fallback to the local model.
- **Graph explosion:** caps, indexes and a logged budget.
- **Confidentiality:** a fully local profile.
- **Wrong hierarchy:** arbitration flag when in doubt.
- **Undetected derogations:** every derogation listed in the report.

4.11 Summary: six structuring decisions

1. The graph serves targeting, not storage.
2. Structured extraction is the central investment.
3. The cross-document bridge is indispensable, and must stay traceable.

4. A QHSE document lives in a document system: versions, derogations and hierarchy belong in the graph.
5. The language model has a narrow place, bound to verified evidence.
6. Targeting recall is the pilot metric.

Chapter 5

Evaluation and Results

This chapter reports what the proof of concept measured on the annotated corpus. The remote profile is the reference; the local profile appears as a comparison column in the tables.

5.1 Experimental setup

Table 5.1: Corpus and ground truth

Document	Code	Type	Level	Clauses
D1	PR-QSE-04	Procedure	3	41
D2	POL-SEC-01	Safety policy	1	37
Total, both written in French				78
Annotated inconsistencies (ground truth)				19
Counter-examples: coherent pairs that look like conflicts				9
Derogations in force at the reference date				1

The corpus was written manually, with injected contradictions (Section 2.4). The level 3 procedure derives from the level 1 policy, so the hierarchy rule of Section 4.4.5 applies.

Table 5.2: Run conditions

Condition	Value
Reference profile	Remote: judge <code>llama-3.3-70b-versatile</code> , through Groq
Comparison profile	Local: judge <code>saiga_mistral_7b_gguf</code> , served by LM Studio
Difference between profiles	Only the stage C judge: same extraction, same pairs, same prompt
Tools	The stack of Table 2.3

Only the judge changes between the two profiles, so any gap in their results comes from stage C.

5.2 Metrics

A detection is a merged finding after the consolidation step (Section 4.7).

Table 5.3: Evaluation metrics

Metric	In plain words	Formula
Targeting recall	Share of the annotated inconsistencies still present after targeting	$\frac{\text{inconsistencies kept}}{N}$
Reduction rate	Share of the cross-document pairs that targeting removes	$1 - \frac{\text{candidate pairs}}{n_1 \times n_2}$
True positive (TP)	A finding with the same clause pair as an annotated inconsistency	
False positive (FP)	A finding that matches no annotated inconsistency	
Precision (P)	Share of the findings that are correct	$\frac{TP}{TP + FP}$
Recall (R)	Share of the annotated inconsistencies that are found	$\frac{TP}{N}$
F1	Balance of precision and recall (harmonic mean)	$\frac{2PR}{P + R}$

where N is the total number of annotated inconsistencies in the ground truth, and n_1, n_2 are the numbers of clauses in the two documents.

5.3 The graph built on the corpus

Table 5.4: Nodes of the graph built on the corpus

Node	Count	What it is
Document	2	A source document
Section	20	A numbered section of a document
Clause	78	A single rule, deadline or responsibility
Concept	331	A term of the domain vocabulary
Actor	18	A recognised QHSE role
Quantity	31	A measured value: deadline, threshold, frequency
Condition	23	A condition that limits when a clause applies
Standard	2	An external standard cited by a clause
Total	505	

Table 5.5: Relationships of the graph built on the corpus

Relationship	Count	What it links
MENTIONS	511	A clause to a concept it talks about
CONTAINS	98	A document to its sections, a section to its clauses
FOLLOWS	76	A clause to the next one, in reading order
CARRIES	31	A clause to a value it sets
UNDER_CONDITION	28	A clause to the condition that limits it
ALIAS_OF	21	Two terms that mean the same thing across the two documents
CITES_STANDARD	2	A clause to an external standard
Total	776	

Only 21 of the 776 relationships, under 3%, link the two documents to each other: all of them are ALIAS_OF. Without them the graph would split into two separate parts, and no cross-document comparison would be possible.

5.4 Targeting results

Table 5.6: Targeting results

Measure	Value
Cross-document pairs (41×37)	1 517
Candidate pairs after targeting	72
Reduction rate ($1 - 72/1\,517$)	95%
Targeting recall	19/19

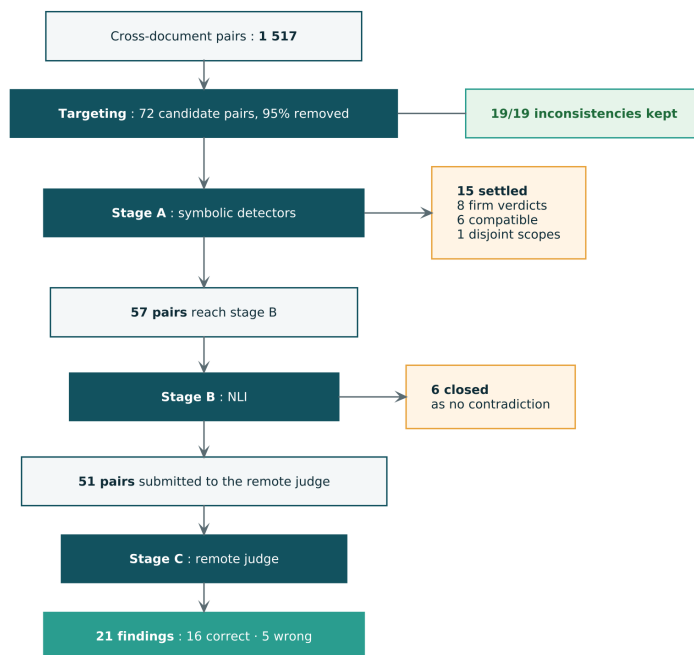


Figure 5.1: Measured flow from the pair space to the findings (remote profile)

Targeting keeps 72 of the 1 517 possible pairs, and all 19 inconsistencies survive it (Figure 5.1). No case is lost at targeting: every missed case is lost later, at detection. I18, a single-clause anomaly, counts as kept because its clause is among the clauses analysed: a single-clause case never needs a candidate pair.

5.5 Detection results

Table 5.7: Flow of the 72 candidate pairs through the cascade

Stage	Pairs received	Pairs settled	How
A, symbolic detectors	72	15	8 firm verdicts, 6 closed as compatible, 1 closed for disjoint scopes
B, NLI	57	6	All closed as no contradiction
C, judge	51	51	Judged by the language model

Up to stage C, both profiles process exactly the same pairs.

Table 5.8: Detection results per profile

	Remote (reference)	Local (comparison)
Judge	llama-3.3-70b-versatile	saiga_mistral_7b_gguf
Findings	21	18
True positives	16	14
False positives	5	4
Precision	0.76	0.78
Recall	0.84	0.74
F1	0.80	0.76

The remote profile finds 16 of the 19 inconsistencies, with 5 false positives (precision 0.76, F1 0.80). It is the reference profile because it measures best: it finds two more inconsistencies than the local profile (I03 and I05) for only one more false positive. The local profile is the confidential alternative: same pipeline, no data leaves the machine, lower recall.

5.6 Cost and reliability

Table 5.9: Cost and reliability of the judge

Measure	Remote (reference)	Local (comparison)
Pairs submitted to the judge	51	51
Format repairs	0	38
Cancelled verdicts (invented proof)	0 (0.0%)	4 (7.8%)
Literal proofs	100%	100%

The remote judge answers all 51 pairs in the expected format, with no repair and no cancelled verdict. Every kept verdict quotes both clauses word for word. For comparison, the local model needed 38 format repairs.

5.7 Effect of the NLI stage

Table 5.10: Effect of stage B on the remote profile

Measure	Without stage B	With stage B
Pairs submitted to the remote judge	57	51
Recall	16/19	16/19
False positives	5	5

On the remote profile, stage B closes 6 of the 57 pairs it receives, all as no contradiction: it only closes pairs, it never asserts one. The remote judge therefore receives 10.5% fewer pairs, while recall (16/19) and false positives (5) stay the same. Stage B lowers the cost of the judge without changing what the system finds.

5.8 Undetected inconsistencies

Table 5.11: Inconsistencies missed by the remote profile

Case	Type	Description	Detector required	Implemented	Cause
I07	Relation	The prevention plan is approved by the QSE Coordinator or QSE Manager (D1 §4.1) and by the Site Director (D2 §4.1)	A4, responsibility conflicts	No	A4 not built; D1 states the approver in a table block
I12	Numeric	An anomaly is reported “within 24 hours” (D1 §6.2); a deviation is escalated “within the week” (D2 §6.2)	A2, numeric divergence	Yes	No shared word, every term below the alias threshold; “within the week” is a vague duration
I18	Derogation	A derogation “valid until 31 December 2024” is still present on 10 August 2026 (D1 §10.3)	A8, faulty derogations	No	A8 not built; a single-clause anomaly, with no clause pair

Two of the three misses (I07, I18) need detectors that were specified but not built. Only one miss, I12, happens with the detectors actually implemented; it is the hardest case of the corpus. The local profile also misses I03 and I05, which the remote profile finds.

Table 5.12: Detectors specified but not implemented

Detector	What it checks	Example
A4	Two documents give the same task to two different people	One document says the Site Director approves a plan, the other says the QSE Manager does
A8	A derogation is expired, or points to a rule that no longer exists	A derogation valid until 2024 is still in force in 2026

Adding these two detectors would cover both remaining cases that the current system cannot see.

5.9 Limitations

Table 5.13: Limitations of the evaluation

Limitation	Effect
Hand-written QHSE lexicon	Rule-based alignment covers only the terms written into it
Hand-written standards registry	A5 checks only the standards listed in it
Thresholds calibrated on the two evaluated documents, no held-out set	The figures are likely optimistic
Only 3 of the 9 specified detectors implemented (A1, A2, A5)	Two of the three misses (I07, I18)
Two-document test corpus, written with injected contradictions	Small, and cleaner than real documents
Segmentation tuned for reasonably structured .txt files	Other formats are untested
Judge behaviour may change from one model version to the next	Stage C results may not reproduce

These results are feasibility indications on one small corpus, not general performance figures.

5.10 Chapter summary

- Targeting removes 95% of the cross-document pairs and keeps all 19 inconsistencies, so every miss happens at detection.
- The remote profile, the reference, finds 16 of the 19 inconsistencies with 5 false positives: precision 0.76, recall 0.84, F1 0.80.
- The remote judge always respects the output format, and every kept verdict carries two literal proofs.
- Stage B cuts the judge’s workload by 10.5% without changing recall or false positives.
- Two of the three misses need detectors that were specified but not built (A4, A8); only I12 is missed with the implemented detectors.
- The figures are feasibility indications, not general performance.

Chapter 6

Conclusion

This study started from a practical problem. Organisations that follow quality, health, safety and environment standards keep many documents that must agree with each other, yet these documents are written and revised by different people at different times. Finding where two of them disagree is still a matter of careful reading, and it does not scale: the number of possible comparisons grows very fast with the size of the documents, while only a few of them hide a real conflict. Checking everything, even with a machine, buries these few conflicts under false alarms.

After defining what counts as an inconsistency in this domain, the study reviewed the existing approaches. Each one brought something useful, at a price. Formal logic and ontologies give proven answers, but only once the text has been rewritten or modelled by hand. Rule-based methods read ordinary text, but are tied to one language and one kind of document. Similarity search avoids comparing everything, but two sentences can be close in meaning without contradicting each other. Contradiction classifiers are cheap but give no evidence, and large language models read raw text well but are costly on whole documents and do not always quote them exactly. No single approach was affordable, backed by evidence and able to read raw French documents at the same time.

The project therefore combined the strongest property of several approaches instead of choosing one. One finding from the literature weighed in particular: writing a graph of the text into a language model’s prompt improves its answers only slightly. A graph seemed more useful for deciding what to check than for helping a model decide. The resulting architecture, COHERA, follows this idea: a graph of the two documents selects the parts worth comparing, simple rules settle the clear cases, and a language model is kept for the hard ones, its verdict counting only when it quotes both documents word for word.

The proof of concept, tested on two documents written for the study, supports this idea. The graph removed 95% of the comparisons without losing any of the 19 inconsistencies, so the selection step, where a missed pair would be lost for good, did not become the weak point. The system then found 16 of the 19, with 5 false alarms. Every quotation in its report is checked against the text, so a reviewer can verify each finding rather than take it on trust. The same system can also run entirely on a local machine for confidential documents, but it then finds fewer inconsistencies.

These results remain indications of feasibility: the test set is small, it was written for the study, the settings were tuned on it, and only part of the designed checks were built. The main directions for future work are:

- build the remaining checks, starting with conflicting responsibilities and expired deroga-

tions, which explain two of the three missed cases, then those for validity dates and the hierarchy between documents;

- enrich the domain vocabulary, so that more terms are matched between documents written by different teams;
- test the system on larger collections of real documents, with settings tuned on separate documents;
- accept common formats such as Word and PDF.

Bibliography

- [1] Qualipro. Official website. <https://www.qualipro-qms.com/en/>. Consulted 24 September 2026.
- [2] Jierui Li, Vipul Raheja, and Dhruv Kumar. ContraDoc: Understanding self-contradictions in documents with large language models. In *Proceedings of NAACL 2024 (Volume 1: Long Papers)*, 2024.
- [3] Siyi Liu and Dan Roth. Conflicts in texts: Data, implications and challenges, 2025. arXiv:2504.19472.
- [4] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, volume 4963 of *Lecture Notes in Computer Science*. Springer, 2008.
- [5] Object Management Group. Semantics of business vocabulary and business rules (SBVR) specification. <https://www.omg.org/spec/SBVR/>.
- [6] Sayandeep Mitra, Kritika Anand, and Pavan Kumar Chittimalli. Identifying anomalies in SBVR-based business rules using directed graphs and SMT-LIBv2. In *Proceedings of the 20th International Conference on Enterprise Information Systems (ICEIS 2018)*. SCITEPRESS, 2018.
- [7] Alan Khoja, Martin Kölbl, Stefan Leue, and Rüdiger Wilhelmi. Automated consistency analysis for legal contracts. *Artificial Intelligence and Law*, 2025. arXiv:2504.18422.
- [8] W3C OWL Working Group. OWL 2 web ontology language document overview (second edition). W3C Recommendation, 2012.
- [9] Birte Glimm, Ian Horrocks, Boris Motik, Giorgos Stoilos, and Zhe Wang. HermiT: An OWL 2 reasoner. *Journal of Automated Reasoning*, 53(3), 2014.
- [10] Veronika Heimsbakk and Kristian Torkelsen. Using the shapes constraint language for modelling regulatory requirements, 2023. arXiv:2309.02723.
- [11] Weize Guo, Li Zhang, and Xiaoli Lian. Automatically detecting the conflicts between software requirements based on finer semantic analysis, 2021. arXiv:2103.02255.
- [12] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP 2019*, 2019.
- [13] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
- [14] Garima Malik, Mucahit Cevik, Ayse Basar, and Devang Parikh. Supervised semantic

- similarity-based conflict detection algorithm: S3CDA, 2022. arXiv:2206.13690, revised 2025.
- [15] Marie-Catherine de Marneffe, Anna N. Rafferty, and Christopher D. Manning. Finding contradictions in text. In *Proceedings of ACL-08: HLT*, 2008.
- [16] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. A large annotated corpus for learning natural language inference. In *Proceedings of EMNLP 2015*, 2015.
- [17] Adina Williams, Nikita Nangia, and Samuel R. Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of NAACL-HLT 2018*, 2018.
- [18] Alexis Conneau, Ruty Rinott, Guillaume Lample, Adina Williams, Samuel R. Bowman, Holger Schwenk, and Veselin Stoyanov. XNLI: Evaluating cross-lingual sentence representations. In *Proceedings of EMNLP 2018*, 2018.
- [19] Louis Martin, Benjamin Muller, Pedro Javier Ortiz Suárez, Yoann Dupont, Laurent Romary, Éric de la Clergerie, Djamé Seddah, and Benoît Sagot. CamemBERT: a tasty French language model. In *Proceedings of ACL 2020*, 2020.
- [20] Cyrille Delestre and Abibatou Amar. DistilCamemBERT: a distillation of the French model CamemBERT. In *Conférence sur l'Apprentissage automatique (CAp 2022)*, 2022. arXiv:2205.11111.
- [21] Nelvin Tan, Yaowen Zhang, James Asikin Cheung, Fusheng Liu, Yu-Ching Shih, and Dong Yang. Improved evidence extraction and metrics for document inconsistency detection with LLMs, 2026. arXiv:2601.02627.
- [22] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization, 2024. arXiv:2404.16130.
- [23] Yingjian Chen, Haoran Liu, Yinhong Liu, Jinxiang Xie, Rui Yang, Han Yuan, Yanran Fu, Peng Yuan Zhou, Qingyu Chen, James Caverlee, and Irene Li. GraphCheck: Breaking long-term text barriers with extracted knowledge graph-powered fact-checking. In *Proceedings of ACL 2025*, 2025.
- [24] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2009)*, 2009.